Cloud platforms: AWS and Azure

Core concepts side by side on AWS and Azure: identity, networks, DNS and CDN, compute, serverless, managed Kubernetes, security, monitoring and cost.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Cloud fundamentals (14 min)
2. Identity and access management (17 min)
3. Virtual networks (17 min)
4. Compute and storage (16 min)
5. DNS, CDN and global traffic on AWS and Azure (15 min)
6. Serverless: Lambda, Azure Functions and event-driven design (16 min)
7. Managed Kubernetes: EKS and AKS (16 min)
8. Cloud monitoring and logging: CloudWatch and Azure Monitor (15 min)
9. Cloud security, governance and compliance guardrails (16 min)
10. Managed services, reliability and cost (16 min)
11. High availability and disaster recovery in the cloud (15 min)
12. Landing zones and multi-account strategy (14 min)
13. Object storage and data lifecycle (12 min)
14. Connecting networks: peering, VPN and private links (13 min)
15. Debugging cloud problems: permissions, networking and quotas (15 min)
16. FinOps: reading a cloud bill and tagging for ownership (12 min)
17. FinOps: rightsizing and removing waste (12 min)
18. FinOps: commitments, savings plans and spot capacity (12 min)
19. FinOps: Kubernetes cost allocation and resource requests (13 min)
20. FinOps: data transfer, storage tiers and hidden costs (12 min)
21. FinOps: budgets, anomaly alerts and a cost practice (11 min)

Lesson 1. Cloud fundamentals

14 minutes. Learn the shared vocabulary of AWS and Azure: regions, availability zones, the shared responsibility model, service types and how to organize accounts.

Cloud platforms rent you computing by the second instead of selling you servers. The vocabulary differs between providers, but the ideas are the same. Learn them once and you can read either AWS or Azure documentation without starting over.

What you will learn

- Regions, availability zones and why they exist
- IaaS, PaaS and SaaS
- The shared responsibility model
- How accounts and subscriptions organize resources

Regions and availability zones

A **region** is a geographic area with several data centers, such as `eu-west-1` (Ireland) in AWS or `westeurope` in Azure. An **availability zone (AZ)** is one or more separate data centers within a region, with independent power and networking, connected by fast links.

Concept	Purpose	Design rule
Region	Latency, data residency, disaster recovery	Choose near users and within legal requirements
Availability zone	Survive a data center failure	Spread each important service across at least two zones

A service running in a single zone is one power failure away from an outage. Running across zones is the basic form of high availability. Surviving the loss of a whole region is a larger, more expensive design that you adopt only when the business needs it.

Service models

Model	You manage	Provider manages	Examples
IaaS	Operating system, runtime, application	Hardware, virtualization	AWS EC2, Azure Virtual Machines
PaaS	Application and data	Everything below, including patching	AWS Elastic Beanstalk, Azure App Service, managed databases
SaaS	Your usage and data	The whole application	Email, ticketing, CRM products

Moving toward PaaS reduces operational work and risk, at the cost of some control. A good default is the most managed service that meets your requirements.

Shared responsibility

The provider secures the cloud itself: buildings, hardware, hypervisors, the global network. **You secure what you put in it**: identities and permissions, data, network rules, operating system patches on virtual machines, and configuration.

> **Why it matters:** *Most publicly reported cloud breaches involve customer configuration, such as a storage bucket opened to the internet or an access key committed to a shared location, not a failure of the provider's data centers.*

MARGA by EknathaLabs
eknathalabs.com

Organizing accounts

	AWS	Azure
Top-level grouping	Organization	Tenant (Microsoft Entra ID) and management groups
Billing and isolation boundary	Account	Subscription
Logical folder inside	Tags, resource groups (optional)	Resource group (required)
Policy at scale	Service control policies	Azure Policy

Use **separate accounts or subscriptions** for production and non-production, and often per team or workload. A boundary at this level limits the blast radius of mistakes and of compromised credentials far better than any tag could.

Cost basics

Cloud billing rewards switching things off. The main levers:

- **Right-size**: match instance sizes to measured usage
- **Commit**: reserved capacity or savings plans for steady workloads
- **Spot or low-priority capacity** for interruptible jobs
- **Delete** unattached disks, old snapshots and idle load balancers
- **Tag** resources with owner and environment, so you can tell whose bill is whose

Set a **budget alert** on every account on day one.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Cloud fundamentals fits together

Part	What it does	What goes wrong
Regions and availability zones	A region is a geographic area with several data centers, such as eu-west-1 (Ireland) in AWS or westeurope in Azure.	Placing production in a single availability zone.
Service models	Moving toward PaaS reduces operational work and risk, at the cost of some control.	Assuming the provider secures your configuration.
Shared responsibility	The provider secures the cloud itself: buildings, hardware, hypervisors, the global network.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Organizing accounts	Use separate accounts or subscriptions for production and non-production, and often per team or workload.	Running everything in one account or subscription.
Cost basics	Cloud billing rewards switching things off.	No budget alert and no tagging.

Flow: Regions and availability zones to Service models / Service models to Shared responsibility / Shared responsibility to Organizing accounts / Organizing accounts to Cost basics

Incident: an unexpected bill

Incident: *The monthly invoice is four times higher than usual, and nobody launched anything new.*

Start with where the cost is, not who to blame. In AWS use Cost Explorer grouped by service and by region; in Azure use Cost Analysis grouped by resource group and meter.

```
$ aws ce get-cost-and-usage \
    --time-period Start=2026-10-01,End=2026-10-24 \
    --granularity DAILY --metrics UnblendedCost \
    --group-by Type=DIMENSION,Key=SERVICE
```

MARGA by EknathaLabs marga.eknathalabs.com

The output shows a large increase in data transfer and compute in an unfamiliar region. Unfamiliar region plus compute is a classic sign of leaked credentials being used to mine cryptocurrency. Treat it as a security incident: disable the exposed keys immediately, review activity in CloudTrail (or the Azure Activity Log), terminate unknown resources, and rotate every credential that could have been exposed. Afterward, add controls that would have caught it sooner: budget alerts at low thresholds, a policy denying use of regions you do not operate in, and short-lived credentials instead of long-lived keys.

Common mistakes

- Running everything in one account or subscription
- Placing production in a single availability zone
- No budget alert and no tagging
- Assuming the provider secures your configuration

Check yourself

What is an availability zone?

Answer: One or more separate data centers within a region with independent power and networking

Under the shared responsibility model, who secures the data and permissions you configure?

Answer: You

Which Azure object is the main billing and isolation boundary, similar to an AWS account?

Answer: Subscription

Large unexpected compute charges appear in a region you never use. What is the most likely cause?

Answer: Leaked credentials being used to create resources

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Identity and access management

17 minutes. Control who can do what in AWS and Azure with roles and policies, avoid long-lived keys, and debug access denied errors systematically.

Identity and access management is the foundation of cloud security. Networks can be misconfigured and software can have bugs, but if every identity has exactly the access it needs and no more, the damage from either is limited.

What you will learn

- Users, groups, roles and policies in AWS
- Entra ID, RBAC and managed identities in Azure
- Why roles and short-lived credentials beat access keys
- How to troubleshoot an access denied error

AWS IAM

Concept	Meaning
User	A long-lived identity for a person or program. Avoid for programs
Group	A set of users sharing permissions
Role	An identity that is **assumed** temporarily. Used by services, applications and federated people
Policy	A JSON document that allows or denies actions on resources

A policy states which **actions** are allowed on which **resources**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::acme-uploads/shop/*"
    }
  ]
}
```

Evaluation follows a clear rule: everything is **denied by default**; an explicit `Allow` is needed; an explicit `Deny` always wins. Permission boundaries, service control policies and session policies can further restrict what an identity can do, even when its own policy allows it.

An EC2 instance, a Lambda function or a Kubernetes service account gets permissions by being given a **role**. The credentials it receives are temporary and rotate automatically, so there is no key to leak.

Azure access control

Azure splits identity from authorization:

Concept	Meaning
Microsoft Entra ID	The identity directory for users, groups and applications
Azure RBAC	Role assignments at a scope: management group, subscription, resource group or resource
Managed identity	An identity for an Azure resource, with no secret to manage
Role assignment	Who (principal) + which role + at what scope

MARGA by EknathaLabs marga.eknathalabs.com

Built-in roles include Reader, Contributor and Owner, and many specific ones. Assign the narrowest role at the narrowest useful scope.

```
$ az role assignment create \
    --assignee-object-id "$PRINCIPAL_ID" \
    --assignee-principal-type ServicePrincipal \
    --role "Storage Blob Data Contributor" \
    --scope /subscriptions/$SUB/resourceGroups/shop-prod/providers/Microsoft.Storage/storageAccoun \
    ts/acmeuploads
```

Principles that apply everywhere

- **Least privilege.** Start with nothing and add what is needed; do not start with admin and remove.
- **Prefer roles and managed identities** over access keys and passwords.
- **Require multi-factor authentication** for every human, and protect the root or global administrator account with hardware keys and no daily use.
- **Use groups** for people, so access follows job role rather than individuals.
- **Review regularly.** Remove unused identities and permissions. Both clouds report last-used information.
- **Log everything**: CloudTrail in AWS, Activity Log and Entra sign-in logs in Azure.

> **Warning:** *An access key that appears in a shared location, a build log or an image must be treated as compromised. Deactivate it immediately, then investigate what it did, then replace it with a role.*

Verifying identity and permissions

```
$ aws sts get-caller-identity
{
    "UserId": "AROAEXAMPLE:session-1",
    "Account": "123456789012",
    "Arn": "arn:aws:sts::123456789012:assumed-role/shop-api-role/session-1"
}
$ az account show --query "{subscription:name, user:user.name}"
```

Always check **who you are** before debugging **what you are allowed to do**.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Identity and access management fits together

Part	What it does	What goes wrong
AWS IAM	A policy states which actions are allowed on which resources.	Wildcard actions and resources ("Action": "*") in policies.
Azure access control	Azure splits identity from authorization.	Long-lived access keys stored in applications.
Principles that apply everywhere	Least privilege. Start with nothing and add what is needed; do not start with admin and remove.	Using the root account or a global administrator for daily work.
Verifying identity and permissions	Always check who you are before debugging what you are allowed to do.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: AWS IAM to Azure access control / Azure access control to Principles that apply everywhere / Principles that apply everywhere to Verifying identity and permissions

Incident: AccessDenied when the application reads a bucket

> **Incident:** *The API returns errors. The log says* `AccessDenied: User: arn:aws:sts::123456789012:assumed-role/shop-api-role/i-0abc is not authorized to perform: s3:GetObject on resource: arn:aws:s3:::acme-uploads/shop/logo.png`.

MARGA by EknathaLabs marga.eknathalabs.com

The error message names the identity, the action and the resource, which is everything you need. Work through the layers:

1. Is the identity what you expect? Here the role is `shop-api-role`. Confirm the application is really using it.
2. Does an identity policy allow the action? The earlier policy allows `GetObject` on `shop/*`, which matches this object.
3. Does the bucket have a resource policy with a `Deny`, or an encryption setting requiring access to a KMS key?
4. Do organization-level controls (service control policies or permission boundaries) block it?

```
$ aws iam simulate-principal-policy \
    --policy-source-arn arn:aws:iam::123456789012:role/shop-api-role \
    --action-names s3:GetObject \
    --resource-arns arn:aws:s3:::acme-uploads/shop/logo.png
```

The simulation reports an implicit deny: the policy on the role lists the key prefix `shop/` but the real object is stored at `Shop/logo.png`, and S3 key names are case-sensitive. Correct the resource path in the policy or the application, and the request succeeds. If the cause had been KMS, the error would have mentioned the key. Reading the denied action and resource carefully saves hours.

Predict the access

Common mistakes

- Using the root account or a global administrator for daily work
- Wildcard actions and resources (`"Action": "*"`) in policies
- Long-lived access keys stored in applications
- Granting access to individuals instead of groups

Check yourself

In AWS policy evaluation, what takes precedence?

Answer: An explicit Deny

Why use a role for an application instead of an access key?

Answer: Role credentials are temporary and rotate automatically, so there is no long-lived secret to leak

What three parts make up an Azure role assignment?

Answer: Principal, role and scope

First step when debugging AccessDenied?

Answer: Confirm which identity is making the call and read the denied action and resource

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Virtual networks

17 minutes. Design a VPC or VNet with public and private subnets, route tables, gateways and security rules, and trace why a connection fails.

Every cloud workload lives in a virtual network that you design. The layout decides what is reachable from the internet, what can talk to what internally, and how much a single mistake can expose. It builds directly on the addressing, routing and firewall ideas from the networking module.

What you will learn

- VPC and VNet structure
- Public and private subnets, route tables, gateways
- Security groups, NACLs and network security groups
- A reachability checklist

The building blocks

Idea	AWS	Azure
The network	VPC	Virtual network (VNet)
A slice of it	Subnet (lives in one AZ)	Subnet (spans the zones of the region)
Routing	Route table per subnet	Route table (UDRs) per subnet
Internet access in	Internet gateway	Public IP on the resource
Outbound only from private subnets	NAT gateway	NAT gateway
Connect networks	VPC peering, Transit Gateway	VNet peering, Virtual WAN
Private access to cloud services	VPC endpoints (PrivateLink)	Private endpoints
Instance-level firewall	Security group (stateful)	Network security group on NIC or subnet (stateful)
Subnet-level firewall	Network ACL (stateless)	NSG on a subnet

A standard layout

```
VPC 10.20.0.0/16
  Public subnets   (10.20.0.0/24, 10.20.1.0/24)   one per AZ
     load balancers, NAT gateways
  Private app subnets (10.20.10.0/24, 10.20.11.0/24)
     application servers or Kubernetes nodes
  Private data subnets (10.20.20.0/24, 10.20.21.0/24)
     databases, no route to the internet
```

A subnet is "public" because its route table sends `0.0.0.0/0` to an internet gateway **and** its resources have public addresses. A "private" subnet has no such route; if it must reach the internet for updates it sends `0.0.0.0/0` to a NAT gateway in a public subnet. Databases usually need no internet route at all.

Plan address ranges so that networks you will later connect (other VPCs, on-premises sites) **do not overlap**. Overlapping ranges cannot be peered, and renumbering later is painful.

Security rules

Security groups and NSGs are stateful: if a connection is allowed in, the return traffic is allowed automatically. Reference other groups instead of IP ranges where you can.

```
resource "aws_security_group_rule" "db_from_app" {
  type                     = "ingress"
  security_group_id        = aws_security_group.db.id
  source_security_group_id = aws_security_group.app.id
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
```

MARGA by EknathaLabs marga.eknathalabs.com

```
}
```

This says "the database accepts PostgreSQL traffic from anything in the app group", which stays correct as application servers come and go.

> **Warning:** *Do not open SSH (22) or RDP (3389) to `0.0.0.0/0`. Use a bastion with strict source ranges, or better, a session service that needs no inbound port (AWS Systems Manager Session Manager, Azure Bastion).*

Reachability checklist

When A cannot reach B, check in this order:

1. **Name resolution**: does the name resolve to the address you expect?
2. **Route**: does A's subnet route table have a path to B's range?
3. **Source firewall**: does A's outbound rule allow it? (Defaults usually do.)
4. **Destination firewall**: does B's security group or NSG allow A on that port?
5. **Subnet filters**: do network ACLs or subnet NSGs allow both directions (stateless ACLs need return rules)?
6. **Listener**: is something actually listening on that port on B?

Cloud tools can analyze the path for you: AWS Reachability Analyzer and Azure Network Watcher (connection troubleshoot and IP flow verify).

Incident: the new service cannot reach its database

> **Incident:** *A new application server in subnet `10.20.10.0/24` times out connecting to PostgreSQL at `10.20.20.15:5432`.*

A timeout (not "connection refused") means packets are being dropped before reaching a listener, which points at a route or firewall. Test from the app server:

```
$ nc -zv -w 3 10.20.20.15 5432
nc: connect to 10.20.20.15 port 5432 (tcp) failed: Connection timed out
$ aws ec2 describe-security-groups --group-ids sg-0db123 \
    --query 'SecurityGroups[0].IpPermissions[].{port:FromPort,from:UserIdGroupPairs[].GroupId}'
[
    { "port": 5432, "from": ["sg-0app111"] }
]
```

The database group allows port 5432 only from `sg-0app111`, but the new server was launched with a different group, `sg-0app222`. Attach the correct group (or add a rule referencing the new one), retest with `nc`, and confirm the application connects. Because the rule referenced a group rather than a fixed IP, fixing group membership was enough. A "refused" error instead would have meant traffic arrived but nothing listened, which sends you to the database process rather than the network.

Diagram: A three tier network in the cloud

Part	What it does	What goes wrong
Internet	Users reach the public load balancer.	Open rules beyond 80 and 443 expose more.
Public subnet	Holds load balancers and NAT.	A server here with a public IP is directly exposed.
Private app subnet	Application servers with no public IP.	Missing NAT means they cannot download updates.
Private data subnet	Databases that accept traffic only from the app tier.	A broad rule exposes data.
NAT gateway	Allows outbound traffic from private subnets.	If it fails, patches and external calls stop.

Flow: Internet to Public subnet / Public subnet to Private app subnet / Private app subnet to Private data subnet / Private app subnet to NAT gateway

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

- Overlapping address ranges between networks
- Placing databases in public subnets
- Opening administrative ports to the whole internet
- Forgetting that network ACLs are stateless
- Single NAT gateway or single zone for something that must stay up

Check yourself

What makes a subnet public in AWS?

Answer: A route to an internet gateway and public addresses on its resources

A connection attempt times out. What does that usually indicate?

Answer: Packets are dropped by a route or firewall before reaching a listener

How do private subnet instances reach the internet for updates without being reachable from it?

Answer: A NAT gateway in a public subnet

What is the best way to allow the app tier to reach the database?

Answer: Allow the database's port from the application's security group

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Compute and storage

16 minutes. Choose between virtual machines, containers and serverless, understand object, block and file storage, and protect data with versioning and backups.

Two decisions come up in every design: where the code runs, and where the data lives. Each has several options on both clouds, and choosing well is mostly about matching the workload to the level of management you want to carry.

What you will learn

- The compute options and when each fits
- Autoscaling and load balancing
- Object, block and file storage
- Protecting data from loss and exposure

Compute options

Option	AWS	Azure	Choose when
Virtual machines	EC2	Virtual Machines	You need full OS control or run legacy software
Managed containers	ECS, Fargate	Container Apps, Container Instances	You want containers without running a cluster
Managed Kubernetes	EKS	AKS	You need Kubernetes features and a portable platform
Functions	Lambda	Azure Functions	Short, event-driven work with spiky traffic
Managed web platform	Elastic Beanstalk, App Runner	App Service	A web app with minimal operations

A rule of thumb: pick the **most managed** option that still fits. A function or managed container has less to patch than a virtual machine. Reach for virtual machines when you must, and for Kubernetes when you have enough services and enough people to justify its cost.

Scaling and load balancing

Put instances behind a **load balancer** across at least two zones, and let an **autoscaling group** (AWS) or **scale set** (Azure) add or remove instances based on a metric such as CPU or request count. Health checks remove bad instances automatically.

```
$ aws autoscaling describe-auto-scaling-groups \
    --query 'AutoScalingGroups[].{name:AutoScalingGroupName,min:MinSize,max:MaxSize,desired:Desire \
    dCapacity}'
```

Instances should be **disposable**: built from an image or startup script, holding no unique data, and replaceable at any time. If a server is precious, it is a risk.

Storage types

Type	AWS	Azure	Use for
Object	S3	Blob Storage	Files, backups, static websites, logs, data lakes
Block	EBS	Managed Disks	A single VM's disk, databases on VMs
File	EFS, FSx	Azure Files	Shared file systems mounted by many machines

MARGA by EknathaLabs marga.eknathalabs.com

Object storage is the workhorse: practically unlimited, durable, cheap and accessed over HTTPS. It is not a file system and is not for frequent small edits to one file.

Storage **classes** trade price for access speed. Frequently read data belongs in the standard tier; old logs belong in an infrequent-access or archive tier, moved by a **lifecycle rule** automatically.

```
resource "aws_s3_bucket_lifecycle_configuration" "logs" {
  bucket = aws_s3_bucket.logs.id

  rule {
    id     = "archive-then-expire"
    status = "Enabled"

    filter {}

    transition {
      days          = 30
      storage_class = "STANDARD_IA"
    }

    expiration {
      days = 365
    }
  }
}
```

Protecting data

- **Block public access** at the account level for object storage, and allow exceptions deliberately.
- **Encrypt at rest** with provider-managed or customer-managed keys. Encrypt in transit with TLS.
- **Turn on versioning**, so an overwritten or deleted object can be recovered.
- **Back up** to a separate account or subscription and **test restores**.
- **Replicate** across zones (default for most managed storage) and, where required, across regions.
- **Snapshot** disks before risky changes.

> **Warning:** *Replication copies mistakes as faithfully as it copies data. If someone deletes a file, replication deletes it everywhere. Versioning and backups are what bring it back.*

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Compute and storage fits together

Part	What it does	What goes wrong
Compute options	A rule of thumb: pick the most managed option that still fits.	Treating a virtual machine as a pet that cannot be rebuilt.
Scaling and load balancing	Put instances behind a load balancer across at least two zones, and let an autoscaling group (AWS) or scale set (Azure) add or remove instances based on a metric such as...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Storage types	Object storage is the workhorse: practically unlimited, durable, cheap and accessed over HTTPS.	Leaving old snapshots and unattached disks to accumulate cost.
Protecting data	Block public access at the account level for object storage, and allow exceptions deliberately.	Confusing replication with backup.

Flow: Compute options to Scaling and load balancing / Scaling and load balancing to Storage types / Storage types to Protecting data

MARGA by EknathaLabs marga.eknathalabs.com

Incident: a bucket is public

> **Incident:** *A security scanner reports that the bucket* `acme-reports` *allows public reads. It contains customer exports.*

Act in this order: close the exposure, then investigate, then fix the cause.

```
$ aws s3api put-public-access-block --bucket acme-reports \
    --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
$ aws s3api get-bucket-policy --bucket acme-reports
$ aws s3api get-bucket-acl --bucket acme-reports
```

Blocking public access takes effect immediately and does not delete data. Then find out how long it was open and what was read: check S3 server access logs or CloudTrail data events, if they were enabled. If you had no logging, you cannot prove nothing was accessed and may need to treat the data as disclosed, which can carry legal notification duties. Find the root cause (a policy with `"Principal": "*"`, a legacy ACL, a deployment that ignored the account-level block) and add controls: account-wide public access block, a policy check in the pipeline, and alerting on any change to bucket exposure.

Common mistakes

- Treating a virtual machine as a pet that cannot be rebuilt
- No versioning or tested restore
- Confusing replication with backup
- Leaving old snapshots and unattached disks to accumulate cost

Check yourself

Which storage type is best for storing backups and static files accessed over HTTPS?

Answer: Object storage

Why is replication not a backup?

Answer: It copies deletions and corruption to the replicas as well

What does a lifecycle rule do?

Answer: Moves or deletes objects automatically based on age

Which is usually the most managed compute option for short, event-driven work?

Answer: Functions

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. DNS, CDN and global traffic on AWS and Azure

15 minutes. Host DNS zones with Route 53 and Azure DNS, choose routing policies, put a CDN in front of applications, and manage certificates and cache behaviour.

Users reach your application through a name, and often through a content delivery network before they ever touch your servers. Getting DNS, caching and certificates right makes a site faster and far more resilient, while getting them wrong causes outages that are hard to see from inside the cloud.

What you will learn

- Managed DNS: hosted zones, record types and alias records
- Routing policies for failover and latency
- What a CDN does and how caching is controlled
- TLS certificates at the edge and common pitfalls

Managed DNS

Route 53 on AWS and **Azure DNS** (or Azure Traffic Manager for routing) host the authoritative records for your domain. You create a zone, then point the registrar's name servers at the zone's name servers.

```
$ aws route53 list-hosted-zones --query 'HostedZones[].[Name,Id]' --output text
$ aws route53 list-resource-record-sets --hosted-zone-id Z0123456789ABC --query 'ResourceRecordSet \
    s[?Type==`A`].[Name,AliasTarget.DNSName]' --output text
$ az network dns zone list --query '[].name' -o tsv
$ az network dns record-set a list -g rg-dns -z example.com -o table
$ dig +short NS example.com
$ dig +short shop.example.com
```

Apex domains (`example.com`) cannot be a CNAME. Route 53 and Azure DNS offer **alias records** that behave like a CNAME at the apex and point to a load balancer, CDN distribution or storage endpoint, with no extra query cost.

Records worth knowing

Type	Use
A / AAAA	Name to IPv4 / IPv6 address
CNAME	Name to another name (not at the apex)
ALIAS (cloud-specific)	Apex or name to a cloud resource
MX	Mail servers
TXT	Verification, SPF, DKIM, DMARC
CAA	Which certificate authorities may issue

TTL controls caching. Lower it to 60 seconds **before** a planned migration, so the change takes effect quickly, and raise it again afterwards.

Routing policies

Beyond a simple answer, DNS can steer traffic:

- **Weighted**: send 5 percent of traffic to a new stack for a canary
- **Latency-based**: answer with the region closest to the user
- **Failover**: answer with a standby when a health check on the primary fails
- **Geolocation**: answer by the user's country, for compliance or content

```
resource "aws_route53_record" "shop_primary" {
  zone_id        = aws_route53_zone.main.zone_id
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  name           = "shop.example.com"
  type           = "A"
  set_identifier = "eu-primary"

  failover_routing_policy {
    type = "PRIMARY"
  }

  health_check_id = aws_route53_health_check.eu.id

  alias {
    name                   = aws_lb.eu.dns_name
    zone_id                = aws_lb.eu.zone_id
    evaluate_target_health = true
  }
}
```

Health-check-driven failover depends on honest health checks. A check that only tests the load balancer will keep sending users to a region whose application is broken.

CDN: CloudFront and Azure Front Door

A **content delivery network** has many edge locations that cache responses near users and terminate TLS there. It reduces latency, offloads your origin and absorbs spikes and some attacks. Azure offers Front Door and Azure CDN; AWS offers CloudFront.

Caching is controlled by headers from the origin and by CDN rules:

```
$ curl -sI https://shop.example.com/static/app.js | grep -iE "cache-control|etag|x-cache|age|via"
cache-control: public, max-age=31536000, immutable
x-cache: Hit from cloudfront
age: 4821
$ curl -sI https://shop.example.com/api/orders | grep -iE "cache-control|x-cache"
cache-control: no-store
x-cache: Miss from cloudfront
```

Cache static assets for a long time with **versioned file names** (`app.3f9a1.js`), so a new release uses a new URL and nothing needs purging. Never cache personalised or authenticated responses unless the cache key includes what makes them different. Purging (invalidation) exists but is slow and sometimes billed, so design to avoid it.

Certificates and the edge

Use managed certificates (AWS Certificate Manager, Azure-managed certificates) that renew automatically. CloudFront requires the certificate in the `us-east-1` region. Set CAA records, enforce HTTPS with a redirect, enable HSTS once you are certain everything serves HTTPS, and keep the origin reachable only from the CDN using a shared secret header or private connectivity.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How DNS, CDN and global traffic on AWS and Azure fits together

Part	What it does	What goes wrong
Managed DNS	Route 53 on AWS and Azure DNS (or Azure Traffic Manager for routing) host the authoritative records for your domain.	Using a CNAME at the zone apex.
Records worth knowing	TTL controls caching. Lower it to 60 seconds before a planned migration, so the change takes effect quickly, and raise it again afterwards.	Caching personalised pages at the CDN.
Routing policies	Beyond a simple answer, DNS can steer traffic.	Failover that depends on a shallow health check.
CDN: CloudFront and Azure Front Door	A content delivery network has many edge locations that cache responses near users and terminate TLS there.	Changing DNS with a long TTL and no plan to wait.
Certificates and the edge	Use managed certificates (AWS Certificate Manager, Azure-managed certificates) that renew automatically.	Letting the origin accept traffic directly, bypassing the CDN and its protections.

Flow: Managed DNS to Records worth knowing / Records worth knowing to Routing policies / Routing policies to CDN: CloudFront and Azure Front Door / CDN: CloudFront and Azure Front Door to Certificates and the edge

Incident: the migration that took a day to complete

> **Incident:** *A team cuts* `shop.example.com` *over to a new load balancer by editing the record. Some customers reach the new stack immediately, others still hit the old one eight hours later, and the old stack has already been shut down.*

The record had a TTL of 86400 seconds, so resolvers cached the old answer for up to a day.

```
$ dig +noall +answer shop.example.com
shop.example.com. 83100 IN A 198.51.100.10
$ dig +noall +answer shop.example.com @ns-123.awsdns-45.com
shop.example.com. 86400 IN A 203.0.113.50
```

The authoritative server already returns the new address, while resolvers hold the old one. Restore the old backend while the cache expires. Next time, lower the TTL to 60 seconds a full TTL-period before the change, make the switch, verify with several resolvers, and only decommission the old stack after the original TTL has passed.

Common mistakes

- Changing DNS with a long TTL and no plan to wait
- Using a CNAME at the zone apex
- Caching personalised pages at the CDN
- Failover that depends on a shallow health check
- Letting the origin accept traffic directly, bypassing the CDN and its protections

Check yourself

Why can't you use a CNAME at the zone apex?

Answer: The DNS standard requires other records (SOA, NS) at the apex, so use an alias record

What should you do with the TTL before a planned DNS migration?

Answer: Lower it in advance so the change propagates quickly

Which technique avoids needing cache invalidation?

Answer: Versioned asset file names, so each release uses a new URL

What does a failover routing policy need to work well?

Answer: A health check that truly reflects whether the application is healthy

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Serverless: Lambda, Azure Functions and event-driven design

16 minutes. Run code without managing servers, connect functions to queues, storage and APIs, understand cold starts, concurrency and limits, and control cost and failures.

Serverless does not mean no servers. It means you hand the provider the job of provisioning, patching and scaling, and pay for execution rather than idle time. It fits spiky, event-driven work very well and some steady, latency-sensitive work very badly, so you need to know the trade-offs.

What you will learn

- The function model on AWS Lambda and Azure Functions
- Triggers: HTTP, queues, storage events and schedules
- Cold starts, concurrency, timeouts and memory settings
- Failure handling: retries, dead-letter queues and idempotency

The model

You upload a small unit of code with a handler. The platform runs it on demand in response to an **event**, scales the number of concurrent copies automatically, and bills by invocation count and duration multiplied by memory size.

```
import json
import os
import boto3

s3 = boto3.client("s3")
BUCKET = os.environ["REPORT_BUCKET"]

def handler(event, context):
    for record in event["Records"]:
        body = json.loads(record["body"])
        key = f"reports/{body['order_id']}.json"
        s3.put_object(Bucket=BUCKET, Key=key, Body=json.dumps(body).encode())
    return {"processed": len(event["Records"])}
```

Initialise clients **outside** the handler, as above, so warm invocations reuse connections instead of rebuilding them.

Concept	AWS	Azure
Function service	Lambda	Azure Functions
HTTP front door	API Gateway or function URL	HTTP trigger, API Management
Queue trigger	SQS	Storage Queue or Service Bus
Event routing	EventBridge	Event Grid
Workflow	Step Functions	Durable Functions, Logic Apps
Identity	Execution role	Managed identity

Triggers

- **HTTP**: a request through a gateway invokes the function synchronously
- **Queue**: messages are read in batches, with the platform scaling consumers with queue depth
- **Storage events**: an object upload invokes a function to create thumbnails or scan files
- **Schedule**: a cron expression runs maintenance tasks

```
$ aws lambda create-function --function-name order-worker --runtime python3.12 --handler app.handl \
    er --zip-file fileb://function.zip --role arn:aws:iam::123456789012:role/order-worker
$ aws lambda invoke --function-name order-worker --payload '{"Records":[]}' --cli-binary-format ra \
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    w-in-base64-out out.json
$ aws lambda get-function-configuration --function-name order-worker --query '[MemorySize,Timeout, \
    Runtime]'
$ aws logs tail /aws/lambda/order-worker --since 15m
$ func start
$ az functionapp list -o table
```

Cold starts, concurrency and limits

A **cold start** happens when the platform must create a new execution environment: download code, start the runtime, run initialisation. It adds latency from tens of milliseconds to a few seconds, depending on language, package size and VPC attachment. Mitigations are smaller packages, lighter runtimes, provisioned concurrency (AWS) or the Premium plan (Azure), and keeping initialisation lean.

Limits matter. A function has a maximum duration (15 minutes on Lambda), a memory range, payload size limits and an **account concurrency limit** shared across all functions. A burst can exhaust it and starve unrelated functions, so set **reserved concurrency** on critical functions and a cap on noisy ones. Memory also sets CPU: giving a function more memory often makes it finish faster, sometimes at a lower total cost.

Failure handling

Queue and event triggers retry on error, so your code runs **at least once**. It must be safe to run twice.

```
def handler(event, context):
    for record in event["Records"]:
        body = json.loads(record["body"])
        if already_processed(body["order_id"]):
            continue
        process(body)
        mark_processed(body["order_id"])
```

Configure a **dead-letter queue** so messages that fail repeatedly are kept for inspection rather than retried forever or dropped. Set the queue's visibility timeout longer than the function's timeout, or messages will be delivered again while the first attempt is still running.

Serverless cost thinking

Cost scales with usage, which is wonderful at low volume and can become the expensive choice at constant high volume, where a container or VM may be cheaper. Watch for a recursive trigger (a function that writes to the bucket that triggers it), unbounded retries, and over-provisioned memory.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Serverless: Lambda, Azure Functions and... fits together

Part	What it does	What goes wrong
The model	You upload a small unit of code with a handler.	Creating clients inside the handler on every call.
Triggers	HTTP: a request through a gateway invokes the function synchronously.	No concurrency limits, letting one function starve others.
Cold starts, concurrency and limits	A cold start happens when the platform must create a new execution environment: download code, start the runtime, run initialisation.	Triggering a function from its own output.
Failure handling	Queue and event triggers retry on error, so your code runs at least once.	Assuming exactly-once delivery.
Serverless cost thinking	Cost scales with usage, which is wonderful at low volume and can become the expensive choice at constant high volume, where a container or VM may be cheaper.	Choosing serverless for steady heavy load without comparing cost.

MARGA by EknathaLabs
marga.eknathalabs.com

Flow: The model to Triggers / Triggers to Cold starts, concurrency and limits / Cold starts, concurrency and limits to Failure handling / Failure handling to Serverless cost thinking

Incident: the function that triggered itself

> **Incident:** *A function resizes images uploaded to a bucket and writes the result back to the same bucket. Within minutes the invocation count explodes to millions, the bill alarm fires, and the account's concurrency limit throttles every other function, including checkout.*

Writing to the trigger bucket created a loop. Break it quickly, then fix the design:

```
$ aws lambda put-function-concurrency --function-name image-resizer --reserved-concurrent-executio \
    ns 0
$ aws s3api get-bucket-notification-configuration --bucket acme-uploads
$ aws lambda delete-function-event-source-mapping --uuid 1234abcd
```

Setting reserved concurrency to zero stops execution immediately. Then write outputs to a **different bucket or prefix**, filter the trigger by prefix (`uploads/`), set budget alarms and concurrency caps on every function, and use a dead-letter queue.

Common mistakes

- Creating clients inside the handler on every call
- Assuming exactly-once delivery
- No concurrency limits, letting one function starve others
- Triggering a function from its own output
- Choosing serverless for steady heavy load without comparing cost

Check yourself

What is a cold start?

Answer: Extra latency while the platform creates a new execution environment

Why must serverless handlers be idempotent?

Answer: Queue and event triggers deliver at least once, so retries and duplicates occur

What does a dead-letter queue hold?

Answer: Messages that repeatedly failed processing, kept for inspection

How can one runaway function affect others?

Answer: It can exhaust the account's shared concurrency limit

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Managed Kubernetes: EKS and AKS

16 minutes. Create and operate managed Kubernetes on AWS and Azure, connect workloads to cloud identity, expose them through load balancers, and plan upgrades and node groups.

Running your own control plane is real work. EKS and AKS take over the API server, etcd and scheduler, so you manage worker nodes and workloads. The skill is knowing where the provider's responsibility ends and what you must still configure well: identity, networking, upgrades and cost.

What you will learn

- What the provider manages and what you manage
- Creating a cluster and connecting `kubectl`
- Workload identity: giving Pods cloud permissions without keys
- Load balancers, node pools and upgrades

Shared responsibility

Area	Provider	You
Control plane availability and patching	Yes	
etcd backups	Yes	
Worker node OS patching	Optional (managed node groups, auto-upgrade)	Choose and enable
Kubernetes version upgrades	Provides versions	Plan and run them
Pod security, RBAC, network policy		Yes
Application deployment and scaling		Yes

Managed does not mean hands-off. Versions go out of support, and an unsupported control plane is eventually upgraded for you, on the provider's schedule.

Creating and connecting

```
$ eksctl create cluster --name shop-prod --region eu-west-1 --version 1.31 --nodegroup-name genera \
    l --node-type m6i.large --nodes 3 --nodes-min 3 --nodes-max 8 --managed
$ aws eks update-kubeconfig --name shop-prod --region eu-west-1
$ kubectl get nodes -o wide
$ az aks create --resource-group rg-shop --name shop-prod --node-count 3 --enable-managed-identity \
     --enable-oidc-issuer --enable-workload-identity --network-plugin azure
$ az aks get-credentials --resource-group rg-shop --name shop-prod
$ kubectl get nodes
```

For production, define clusters in Terraform rather than one-off commands, so they are reviewed, repeatable and reproducible.

Workload identity

A Pod that needs S3, a queue or a key vault should not carry access keys. Both clouds can map a Kubernetes **service account** to a cloud identity, so the Pod receives short-lived credentials automatically.

On EKS this is **IAM Roles for Service Accounts** (or EKS Pod Identity); on AKS it is **Workload Identity**.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders-worker
  namespace: shop
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/shop-orders-worker
---
apiVersion: apps/v1
kind: Deployment
metadata:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  name: orders-worker
  namespace: shop
spec:
  replicas: 2
  selector:
    matchLabels:
      app: orders-worker
  template:
    metadata:
      labels:
        app: orders-worker
    spec:
      serviceAccountName: orders-worker
      containers:
        - name: worker
          image: registry.example.com/shop/orders-worker:2.1.0
```

The IAM role's trust policy allows only that exact service account in that exact cluster. Scope each role to what one application needs. Granting node-level permissions instead gives every Pod on the node the same access, which is far too broad.

Exposure and networking

A Service of type `LoadBalancer` makes the cloud create a load balancer. An Ingress controller (the AWS Load Balancer Controller, or the Application Gateway or NGINX ingress on Azure) shares one load balancer across many hosts.

```
$ kubectl get svc -n ingress-nginx
$ kubectl get ingress -A
$ kubectl describe svc shop-api -n shop | grep -iE "loadbalancer|events" -A3
```

With cloud-native CNIs (AWS VPC CNI, Azure CNI) every Pod uses a real address from the subnet, so a large cluster can exhaust a small subnet. Plan CIDR ranges before creation. Use private clusters or restrict the API endpoint to trusted networks, and use security groups or NSGs and network policies for traffic control.

Nodes and upgrades

Use **managed node groups** (EKS) or **node pools** (AKS) with several instance types and zones. Keep system and user workloads in separate pools, use spot or low-priority capacity only for interruption-tolerant work, and attach a cluster autoscaler or Karpenter.

```
$ eksctl upgrade cluster --name shop-prod --version 1.31 --approve
$ eksctl upgrade nodegroup --name general --cluster shop-prod --kubernetes-version 1.31
$ az aks get-upgrades --resource-group rg-shop --name shop-prod -o table
$ az aks upgrade --resource-group rg-shop --name shop-prod --kubernetes-version 1.31.2
```

Upgrade one minor version at a time, staging first. Check deprecated API usage before upgrading.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Managed Kubernetes: EKS and AKS fits together

Part	What it does	What goes wrong
Shared responsibility	Managed does not mean hands-off. Versions go out of support, and an unsupported control plane is eventually upgraded for you, on the provider's schedule.	Ignoring Kubernetes version support deadlines.
Creating and connecting	For production, define clusters in Terraform rather than one-off commands, so they are reviewed, repeatable and reproducible.	Putting broad permissions on the node role.
Workload identity	A Pod that needs S3, a queue or a key vault should not carry access keys.	Treating the cluster as a pet created by hand instead of code.
Exposure and networking	A Service of type LoadBalancer makes the cloud create a load balancer.	Choosing a small subnet and running out of Pod addresses.
Nodes and upgrades	Use managed node groups (EKS) or node pools (AKS) with several instance types and zones.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Shared responsibility to Creating and connecting / Creating and connecting to Workload identity / Workload identity to Exposure and networking / Exposure and networking to Nodes and upgrades

Incident: the Pod that could read every bucket

> **Incident:** *A security review finds that a small internal tool, running on the cluster, can list and read every S3 bucket in the account. It uses no access keys. A penetration tester shows that anyone who gets a shell in any Pod can read customer exports.*

The Pods inherited the **node instance profile**, which had broad S3 access to support an old batch job.

```
$ kubectl exec deploy/internal-tool -n tools -- aws sts get-caller-identity
$ kubectl exec deploy/internal-tool -n tools -- env | grep -E "AWS_ROLE_ARN|AWS_WEB_IDENTITY"
```

No web identity variables means the Pod falls back to the node's role. Fix it by giving each workload its own service account and role, reducing the node role to the minimum the kubelet needs, and blocking Pod access to the instance metadata service (set the hop limit to 1 and require IMDSv2). Verify with `sts get-caller-identity` from a test Pod.

Common mistakes

- Putting broad permissions on the node role
- Choosing a small subnet and running out of Pod addresses
- Public API endpoint open to the world
- Ignoring Kubernetes version support deadlines
- Treating the cluster as a pet created by hand instead of code

Check yourself

What does a managed Kubernetes service run for you?

Answer: The control plane, including the API server and etcd

What is the purpose of IRSA or Workload Identity?

Answer: Give Pods their own short-lived cloud credentials mapped to a service account

Why plan subnet sizes for VPC-native CNIs?

Answer: Each Pod consumes a real address, so a small subnet limits cluster size

How should you upgrade a managed cluster?

Answer: One minor version at a time, testing in staging first

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Cloud monitoring and logging: CloudWatch and Azure Monitor

15 minutes. Collect metrics, logs and audit trails with CloudWatch, CloudTrail, Azure Monitor and Log Analytics, query them, build alarms and keep retention and cost under control.

Every cloud service emits telemetry, but nothing reads it for you. Production needs three things from the platform: **metrics** that show health and capacity, **logs** that explain behaviour, and **audit trails** that record who changed what. Knowing the native tools is the foundation even if you later add Prometheus or a commercial platform.

What you will learn

- Metrics, logs and audit trails on each cloud
- Querying logs with CloudWatch Logs Insights and KQL
- Creating alarms that notify the right people
- Retention, cost and cross-account collection

Which service does what

Purpose	AWS	Azure
Metrics and alarms	CloudWatch Metrics, Alarms	Azure Monitor Metrics, Alerts
Application and system logs	CloudWatch Logs	Log Analytics workspace
Audit of API actions	CloudTrail	Activity Log
Tracing	X-Ray	Application Insights
Dashboards	CloudWatch Dashboards	Workbooks, dashboards

Metrics are numbers over time with short retention at full resolution. Logs are text events and are far more expensive to store and search. Audit trails are your security record, and must be protected from tampering.

Metrics and alarms

```
$ aws cloudwatch list-metrics --namespace AWS/ApplicationELB --metric-name TargetResponseTime | he \
    ad -20
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization --dimensio \
    ns Name=InstanceId,Value=i-0abc123 --start-time 2025-06-02T00:00:00Z --end-time 2025-06-02T06: \
    00:00Z --period 300 --statistics Average
$ aws cloudwatch describe-alarms --state-value ALARM --query 'MetricAlarms[].AlarmName'
$ az monitor metrics list --resource /subscriptions/0000/resourceGroups/rg-shop/providers/Microsof \
   t.Compute/virtualMachines/web-1 --metric "Percentage CPU" --interval PT5M
$ az monitor metrics alert list -g rg-shop -o table
```

A good alarm reflects user impact or imminent failure, has a clear owner, and links to a runbook.

```
resource "aws_cloudwatch_metric_alarm" "alb_5xx" {
  alarm_name          = "shop-alb-5xx-rate"
  namespace           = "AWS/ApplicationELB"
  metric_name         = "HTTPCode_Target_5XX_Count"
  statistic           = "Sum"
  period              = 60
  evaluation_periods  = 5
  datapoints_to_alarm = 3
  threshold           = 20
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data  = "notBreaching"
  alarm_actions       = [aws_sns_topic.oncall.arn]
  ok_actions          = [aws_sns_topic.oncall.arn]
}
```

`datapoints_to_alarm` of 3 out of 5 reduces noise from single spikes. Choose `treat_missing_data` deliberately: for a metric that should always report, missing data should alarm.

MARGA by EknathaLabs marga.eknathalabs.com

Querying logs

Install an agent or enable the platform integration to ship logs, use structured JSON where possible, and learn the query language.

```
fields @timestamp, @message
| filter @message like /ERROR/
| stats count() as errors by bin(5m)
| sort @timestamp desc
| limit 50
```

That is CloudWatch Logs Insights. The Azure equivalent is **Kusto Query Language (KQL)** in Log Analytics:

```
AppTraces
| where TimeGenerated > ago(1h) and SeverityLevel >= 3
| summarize errors = count() by bin(TimeGenerated, 5m), AppRoleName
| order by TimeGenerated desc
```

```
$ aws logs start-query --log-group-name /shop/api --start-time 1748822400 --end-time 1748826000 -- \
    query-string 'fields @timestamp, @message | filter @message like /ERROR/ | limit 20'
$ aws logs tail /shop/api --since 30m --follow
$ az monitor log-analytics query -w 00000000-0000-0000-0000-000000000000 --analytics-query "AppTra \
    ces | take 10"
```

Audit trails

CloudTrail and the Azure Activity Log answer security and troubleshooting questions: who deleted this, when did the security group change, which role was assumed.

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteSec \
    urityGroup --max-results 5
$ az monitor activity-log list --offset 24h --query "[?authorization.action=='Microsoft.Network/ne \
    tworkSecurityGroups/delete']" -o table
```

Enable the trail in **every region and account**, send it to a separate, locked-down log archive account with object lock or immutability, and turn on integrity validation. An attacker's first step is often to disable or delete logging.

Retention and cost

Log ingestion and retention are among the fastest-growing cloud bills.

- Set a retention period on every log group, since the default is often "never expire"
- Sample or drop noisy debug logs in production
- Archive old logs to cheap object storage, and query them with Athena or Azure Data Explorer when needed
- Alert on ingestion volume spikes, since a logging loop can cost thousands of dollars in a weekend

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Cloud monitoring and logging: CloudWatch and... fits together

Part	What it does	What goes wrong
Which service does what	Metrics are numbers over time with short retention at full resolution.	Logs with unlimited retention and no budget alerts.
Metrics and alarms	A good alarm reflects user impact or imminent failure, has a clear owner, and links to a runbook.	Alarms only on CPU and disk, none on errors and latency.
Querying logs	Install an agent or enable the platform integration to ship logs, use structured JSON where possible, and learn the query language.	Unstructured logs that cannot be queried.
Audit trails	CloudTrail and the Azure Activity Log answer security and troubleshooting questions: who deleted this, when did the security group change, which role was assumed.	Audit trail in the same account that attackers can modify.
Retention and cost	Log ingestion and retention are among the fastest-growing cloud bills.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Which service does what to Metrics and alarms / Metrics and alarms to Querying logs / Querying logs to Audit trails / Audit trails to Retention and cost

Incident: the outage nobody was paged for

> **Incident:** *Customers cannot check out for 40 minutes before a support ticket reveals it. All dashboards were green. The alarm on instance CPU never fired, and the only alert on the load balancer watched for zero healthy hosts.*

The service was up but returning errors. Look at what the platform recorded:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB --metric-name HTTPCode_Targe \
    t_5XX_Count --dimensions Name=LoadBalancer,Value=app/shop/abc123 --start-time 2025-06-02T10:00 \
    :00Z --end-time 2025-06-02T11:00:00Z --period 60 --statistics Sum
```

The data shows hundreds of 5xx responses per minute starting at 10:12. Infrastructure alarms measured resources, not user experience. Add alarms on error rate and latency, an alarm on missing data, a synthetic checkout probe from outside, and route alerts to the on-call rotation with a runbook link.

Common mistakes

- Alarms only on CPU and disk, none on errors and latency
- Logs with unlimited retention and no budget alerts
- Audit trail in the same account that attackers can modify
- Unstructured logs that cannot be queried
- Alerts that go to an email nobody watches

Check yourself

Which service records who made API calls in an AWS account?

Answer: CloudTrail

What is KQL?

Answer: The query language used in Azure Monitor Log Analytics

Why set retention on every log group?

Answer: Defaults may keep logs forever, which grows cost without value

What should a good alarm be based on?

Answer: User-visible symptoms such as error rate and latency, as well as imminent resource failure

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Cloud security, governance and compliance guardrails

16 minutes. Organise accounts and subscriptions, enforce guardrails with SCPs and Azure Policy, protect data with encryption and key management, and detect misconfiguration continuously.

The cloud makes it easy to create resources and just as easy to create insecure ones. At scale you cannot review every change by hand. **Governance** means building guardrails so that safe is the default and unsafe is blocked or flagged automatically.

What you will learn

- Account and subscription structure for isolation
- Preventive controls: SCPs, Azure Policy and permission boundaries
- Encryption, key management and secrets
- Detective controls: posture management and findings

Structure for isolation

Do not run everything in one account. Separate accounts (AWS) or subscriptions (Azure) give hard boundaries for blast radius, billing and access.

Level	AWS	Azure
Top	Organization	Tenant and management groups
Unit of isolation	Account	Subscription
Grouping	Organizational Units	Management groups
Guardrail	Service control policies	Azure Policy, RBAC deny assignments

A common layout has separate accounts for production, non-production, shared networking, security tooling and the log archive. A compromise in a development account then cannot touch production data or erase audit logs.

Preventive guardrails

A **service control policy** sets the maximum permissions available in an account, whatever IAM says. Even an administrator inside the account cannot exceed it.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyDisablingAudit",
      "Effect": "Deny",
      "Action": [
        "cloudtrail:StopLogging",
        "cloudtrail:DeleteTrail",
        "config:StopConfigurationRecorder"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyUnapprovedRegions",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "route53:*", "cloudfront:*", "support:*"],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["eu-west-1", "eu-central-1"]
        }
      }
    }
  ]
}
```

MARGA by EknathaLabs marga.eknathalabs.com

On Azure, **Azure Policy** can deny, audit or auto-remediate resources that break rules, such as storage accounts that allow public blob access or resources in the wrong region.

```
$ aws organizations list-policies --filter SERVICE_CONTROL_POLICY
$ aws organizations list-accounts --query 'Accounts[].[Name,Id]' --output text
$ az policy assignment list --query '[].{name:displayName,scope:scope}' -o table
$ az policy state summarize --top 5
```

Roll out guardrails in **audit mode** first, learn what would break, then enforce. A denied deployment with an unclear message creates support tickets, so write error text that explains the rule.

Data protection

- Encrypt data at rest with managed keys, and use customer-managed keys (KMS, Key Vault) where policy requires control and rotation
- Require TLS for data in transit and block plain HTTP on storage endpoints
- Block public access at the account level for object storage
- Keep secrets in Secrets Manager or Key Vault, with access by identity and audit logging
- Separate key administrators from data users

```
$ aws s3control get-public-access-block --account-id 123456789012
$ aws kms list-aliases --query 'Aliases[?starts_with(AliasName, `alias/shop`)].AliasName'
$ aws kms get-key-rotation-status --key-id alias/shop-data
$ az storage account show -n acmedata --query '{https:enableHttpsTrafficOnly, publicAccess:allowBl \
    obPublicAccess, tls:minimumTlsVersion}'
$ az keyvault show -n kv-shop --query '{purge:properties.enablePurgeProtection, soft:properties.en \
    ableSoftDelete}'
```

Detective controls

Prevention will not catch everything. Continuous assessment finds drift and misconfiguration: AWS Security Hub, Config and GuardDuty; Microsoft Defender for Cloud and Azure Policy compliance. Feed findings into a ticketing system with owners and deadlines, and prioritise by exposure: a public database matters more than a missing tag.

Compliance as code

Express controls as code that runs in the pipeline before deployment, for example scanning Terraform with a policy engine, and run the same checks against live accounts. That keeps documentation, enforcement and evidence consistent for audits such as ISO 27001, SOC 2 or PCI DSS.

```
$ terraform show -json tfplan > plan.json
$ trivy config --severity HIGH,CRITICAL .
$ checkov -d . --quiet --compact
$ aws securityhub get-findings --filters '{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUA \
  LS"}],"RecordState":[{"Value":"ACTIVE","Comparison":"EQUALS"}]}' --max-items 5
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Cloud security, governance and compliance... fits together

Part	What it does	What goes wrong
Structure for isolation	Do not run everything in one account. Separate accounts (AWS) or subscriptions (Azure) give hard boundaries for blast radius, billing and access.	One account for everything.
Preventive guardrails	A service control policy sets the maximum permissions available in an account, whatever IAM says.	Guardrails enforced without an audit period or helpful error messages.
Data protection	Encrypt data at rest with managed keys, and use customer-managed keys (KMS, Key Vault) where policy requires control and rotation.	Key administrators who can also read all the data.
Detective controls	Prevention will not catch everything. Continuous assessment finds drift and misconfiguration: AWS Security Hub, Config and GuardDuty; Microsoft Defender for Cloud and...	Relying on documentation instead of automated controls.
Compliance as code	Express controls as code that runs in the pipeline before deployment, for example scanning Terraform with a policy engine, and run the same checks against live accounts.	Findings dashboards that nobody owns.

Flow: Structure for isolation to Preventive guardrails / Preventive guardrails to Data protection / Data protection to Detective controls / Detective controls to Compliance as code

Incident: the public bucket in the dev account

> **Incident:** *A researcher reports that a storage bucket named after your company lists customer invoices to anyone. A developer had created it in a sandbox account for a demo, copied real exports into it, and set a public-read policy to share a link.*

Contain before you investigate:

```
$ aws s3api put-public-access-block --bucket acme-demo-invoices --public-access-block-configuratio \
    n BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
$ aws s3api delete-bucket-policy --bucket acme-demo-invoices
$ aws s3api get-bucket-logging --bucket acme-demo-invoices
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=ResourceName,AttributeValue=acme-d \
    emo-invoices --max-results 20
```

Block access, preserve logs, determine what was downloaded and by whom, and follow your breach-notification process. Prevent a repeat with an account-level public access block enforced by an SCP, a rule that production data may not leave production accounts, and automated detection that alerts within minutes when a bucket becomes public.

Common mistakes

- One account for everything
- Guardrails enforced without an audit period or helpful error messages
- Relying on documentation instead of automated controls
- Key administrators who can also read all the data
- Findings dashboards that nobody owns

Check yourself

What does a service control policy do?

Answer: Sets the maximum permissions available in an account, regardless of IAM policies

Why use separate accounts or subscriptions?

Answer: To limit the blast radius and separate access, billing and audit data

MARGA by EknathaLabs marga.eknathalabs.com

What is a safe way to introduce a new guardrail?

Answer: Run it in audit mode first, review the impact, then enforce

What is the first priority when a data exposure is found?

Answer: Block the public access and preserve evidence

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Managed services, reliability and cost

16 minutes. Use managed databases, queues and DNS well, design for failure across zones, and keep spending visible and under control.

Operating a database yourself means owning backups, patching, failover and tuning. A managed service takes most of that work, and you pay for it in money and some loss of control. Deciding what to run yourself and what to rent is one of the most valuable judgments a cloud engineer makes.

What you will learn

- Common managed services and what they replace
- High availability versus disaster recovery
- Recovery objectives (RTO and RPO)
- A cost-control routine

Managed services worth knowing

Need	AWS	Azure
Relational database	RDS, Aurora	Azure Database for PostgreSQL, MySQL; Azure SQL
Key-value / document store	DynamoDB	Cosmos DB
Cache	ElastiCache	Azure Cache for Redis
Queue and messaging	SQS, SNS, EventBridge	Service Bus, Event Grid
DNS	Route 53	Azure DNS
Content delivery	CloudFront	Azure Front Door, CDN
Secrets	Secrets Manager, Parameter Store	Key Vault
Monitoring	CloudWatch	Azure Monitor

Queues deserve a special mention. Putting a queue between components absorbs bursts, lets a slow consumer catch up, and means a failure in one part does not instantly break the rest. Design consumers to be **idempotent**, because delivery may be repeated.

High availability and disaster recovery

They answer different questions:

	High availability (HA)	Disaster recovery (DR)
Protects against	A component or zone failing	Losing a whole region or a data-destroying event
Typical design	Multiple zones, automatic failover	Backups elsewhere, a second region
Recovery	Automatic, seconds to minutes	Planned, minutes to hours or more

Two numbers define the target for any system:

- **RTO** (recovery time objective): how long can we be down?
- **RPO** (recovery point objective): how much recent data can we afford to lose?

An RPO of 24 hours is met by nightly backups. An RPO of 5 minutes needs continuous replication. Tighter objectives cost much more, so agree on them with the business per system, not as one blanket number. A disaster recovery plan that has never been **exercised** is a hypothesis.

For a managed database, enable Multi-AZ (AWS) or zone-redundant HA (Azure) for availability, automated backups with point-in-time recovery for data protection, and deletion protection to stop an accidental

MARGA by EknathaLabs marga.eknathalabs.com

removal.

Infrastructure that fails gracefully

- **Timeouts and retries with backoff** on every network call
- **Health checks** that remove unhealthy instances
- **Graceful degradation**: if recommendations are down, still show the product page
- **Limit blast radius** with separate accounts, zones and per-service quotas
- **Test failure** deliberately: stop an instance, fail over a database, and see what actually happens

Controlling cost

Practice	Effect
Budgets with alerts at 50%, 80%, 100%	No surprises at month end
Tags: `owner`, `env`, `service`	Costs can be attributed
Review the top five cost items monthly	Most savings sit in a few services
Right-size from measured use	Stops paying for idle capacity
Commitments for steady load	Large discounts for predictable baseline
Shut down non-production out of hours	Often saves more than half of those costs
Delete orphans: disks, snapshots, IPs, load balancers	Removes waste with no impact

```
$ aws ec2 describe-volumes --filters Name=status,Values=available \
    --query 'Volumes[].{id:VolumeId,gb:Size}' --output table
```

Unattached ("available") volumes are paying for storage nobody uses.

Watch data transfer. Traffic between zones, out to the internet and between regions is billed, and chatty cross-zone designs can create surprising charges.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Managed services, reliability and cost fits together

Part	What it does	What goes wrong
Managed services worth knowing	Queues deserve a special mention. Putting a queue between components absorbs bursts, lets a slow consumer catch up, and means a failure in one part does not instantly...	Using IP addresses instead of service DNS names.
High availability and disaster	They answer different questions.	Treating Multi-AZ as a backup.
Infrastructure that fails gracefully	Timeouts and retries with backoff on every network call.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Controlling cost	Unattached ("available") volumes are paying for storage nobody uses.	No cost alerts or ownership tags.

Flow: Managed services worth knowing to High availability and disaster / High availability and disaster to Infrastructure that fails gracefully / Infrastructure that fails gracefully to Controlling cost

Incident: a database failover causes an outage anyway

> **Incident:** *The primary database's zone fails. Multi-AZ failover happens in about a minute, but the application stays down for 25 minutes.*

The database did its job. The application did not:

MARGA by EknathaLabs marga.eknathalabs.com

```
$ kubectl logs deploy/shop-api -n shop --tail 4
ERROR connection refused to 10.20.20.15:5432
ERROR connection refused to 10.20.20.15:5432
ERROR connection refused to 10.20.20.15:5432
```

The application connected using the database's **IP address**, which it cached. After failover the endpoint's DNS name points at the new primary, but the app never re-resolved it. Fixes: connect using the **DNS endpoint** the service provides, never an IP; keep DNS caching time low in the client; make the client reconnect with backoff when connections drop; and run a **failover exercise** on a schedule so you discover this in a test, not at 3 a.m. Also record the real recovery time from the exercise, since it is the honest figure to compare with the agreed RTO.

Common mistakes

- Treating Multi-AZ as a backup
- Using IP addresses instead of service DNS names
- Setting RTO and RPO as numbers nobody has tested
- No cost alerts or ownership tags
- Running self-managed databases without anyone responsible for restores

Check yourself

What does RPO describe?

Answer: How much recent data loss is acceptable

Which feature protects against a single availability zone failing?

Answer: Multi-AZ or zone-redundant deployment with automatic failover

Why should applications connect to a database through its DNS name?

Answer: After failover the name points to the new primary, while an IP address does not

Which action usually removes cost without any user impact?

Answer: Removing unattached disks and old snapshots

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. High availability and disaster recovery in the cloud

15 minutes. Design for failure at the level of instance, zone and region, choose between backup, pilot light, warm standby and active-active, and prove recovery targets with tests.

Everything fails eventually: disks, hosts, whole data centres, sometimes an entire region, and often people (a bad deploy, a wrong command). High availability keeps a service running through small failures automatically. Disaster recovery restores it after large ones. They have different costs and different tests, and you need to choose them on purpose, based on what an hour of downtime costs.

What you will learn

- Failure domains: instance, zone, region, account, provider
- Recovery time objective (RTO) and recovery point objective (RPO)
- The four disaster recovery patterns
- Data replication and its trade-offs
- Testing: restore drills and failover exercises

Failure domains

Domain	Typical failure	Protection
Instance	Hardware or process crash	Auto scaling, health checks, stateless design
Zone	Power or network issue in one data centre	Spread across at least two or three zones
Region	Large regional outage, bad regional dependency	Second region
Account	Compromised or deleted account, billing lock	Backups in a separate account
Human	Wrong delete, bad deploy, ransomware	Backups, versioning, immutable copies, change control

Multi-zone design covers most hardware and data-centre failures cheaply and should be the default. Multi-region design is much more expensive and complex; use it when the business requires it.

RTO and RPO

RTO is how long you can be down. **RPO** is how much data you can lose, measured in time since the last good copy. An RPO of 15 minutes needs backups or replication at least that often. Ask the business what each costs per hour, then pick a design that matches, not the biggest one available.

Disaster recovery patterns

Pattern	What runs in the second region	RTO	RPO	Cost
Backup and restore	Only backups copied over	Hours	Hours	Lowest
Pilot light	Data replicated; minimal core services off or tiny	Tens of minutes	Minutes	Low
Warm standby	Full stack at reduced size	Minutes	Seconds to minutes	Medium
Active-active	Full stack serving traffic in both	Near zero	Near zero	Highest

MARGA by EknathaLabs marga.eknathalabs.com

Data is the hard part

Stateless services are easy to rebuild. Data needs replication: synchronous replication has no loss but adds latency and can stop writes if the link breaks; asynchronous replication is faster but may lose the latest writes. Replication copies mistakes too: a deleted table is deleted everywhere, so keep point-in-time recovery and separate backups. Test restores, because an untested backup is a hope.

Traffic and automation

Use health-checked DNS or a global load balancer to move traffic. Keep DNS TTLs low enough for your RTO. Define everything as code so the second region can be created by a pipeline, not by memory. Keep secrets, container images and configuration available in the second region, and check that quotas there are high enough.

Prove it

```
$ aws rds restore-db-instance-to-point-in-time \
    --source-db-instance-identifier shop-prod \
    --target-db-instance-identifier shop-restore-test \
    --restore-time 2026-10-20T09:00:00Z
```

Run restore drills each quarter and time them. Run a failover exercise (a game day) that actually moves traffic, and record the measured RTO and RPO. Update the runbook with what you learn.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How High availability and disaster recovery in the... fits together

Part	What it does	What goes wrong
Failure domains	Multi-zone design covers most hardware and data-centre failures cheaply and should be the default.	Choosing multi-region without a business reason.
RTO and RPO	RTO is how long you can be down. RPO is how much data you can lose, measured in time since the last good copy.	Treating replication as backup.
Disaster recovery patterns	.	Forgetting quotas, secrets and images in the second region.
Data is the hard part	Stateless services are easy to rebuild. Data needs replication: synchronous replication has no loss but adds latency and can stop writes if the link breaks; asynchronous...	Data needs replication: synchronous replication has no loss but adds latency and can stop writes if the link breaks; asynchronous...
Traffic and automation	Use health-checked DNS or a global load balancer to move traffic.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Prove it	Run restore drills each quarter and time them.	Never testing a restore or failover.

Flow: Failure domains to RTO and RPO / RTO and RPO to Disaster recovery patterns / Disaster recovery patterns to Data is the hard part / Data is the hard part to Traffic and automation / Traffic and automation to Prove it

Incident: the failover that did not work

> **Incident:** *A region becomes unavailable and the team starts a failover that had never been tested. It takes nine hours instead of the planned one.*

```
$ terraform plan -var region=eu-west-1
Error: creating RDS Cluster: DBSubnetGroupDoesNotExist
Error: InstanceLimitExceeded: You have requested more instances than your quota
```

Network pieces were missing in the second region, the instance quota there was too low, and the latest container image had not been replicated. After the event the team replicated images, raised quotas, kept the stack as code that applies cleanly in both regions, and ran a quarterly failover drill with measured

results.

Common mistakes

- Treating replication as backup
- Never testing a restore or failover
- Choosing multi-region without a business reason
- Forgetting quotas, secrets and images in the second region

Check yourself

What does RPO measure?

Answer: How much data can be lost, measured in time

Which pattern keeps a minimal environment ready and data replicated?

Answer: Pilot light

Why is replication not a backup?

Answer: It also copies mistakes and deletions

How do you know your recovery targets are real?

Answer: Run timed restore and failover drills

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Landing zones and multi-account strategy

14 minutes. Organise cloud accounts or subscriptions into a hierarchy with guardrails, central logging, networking and identity, so teams move fast inside safe boundaries.

A single cloud account for everything works at first. Then a test script deletes a production resource, a bill cannot be split between teams, a compromised key reaches everything, and a service limit used by one team blocks another. The answer is not more rules, but more boundaries: separate accounts (AWS), subscriptions (Azure) or projects (Google Cloud), organised under one management structure with shared guardrails.

What you will learn

- Why accounts are the strongest boundary
- A typical hierarchy and account roles
- Guardrails: organisation policies and service control policies
- Central logging, networking and identity
- Vending new accounts with automation

Why many accounts

An account is a boundary for permissions, limits, billing and blast radius. Splitting by environment and workload gives you:

- **Security isolation:** a compromise in development cannot reach production data.
- **Cost clarity:** each account has its own bill and budget.
- **Independent limits:** one team's quota use does not block another.
- **Simpler permissions:** production access can be narrow, development access broad.

A typical structure

```
Organization root
  Security OU
    log-archive          (central, write-only logs)
    security-tooling     (detection, audit read access)
  Infrastructure OU
    network-hub          (shared networking, DNS, egress)
    shared-services      (CI runners, registries)
  Workloads OU
    Prod OU:     app-a-prod, app-b-prod
    Non-prod OU: app-a-dev,  app-a-staging
  Sandbox OU             (experiments, tight budgets, auto-clean)
  Management account     (billing and organisation only; no workloads)
```

Keep workloads out of the management account. Organise by what needs different rules, not by the company's org chart.

Guardrails

Policies attached to organisational units limit what even administrators in an account can do:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyLeavingApprovedRegions",
    "Effect": "Deny",
    "NotAction": ["iam:*", "organizations:*", "support:*", "route53:*", "cloudfront:*"],
    "Resource": "*",
    "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["eu-west-1", "eu-central-1"] } }
  }]
}
```

MARGA by EknathaLabs marga.eknathalabs.com

Typical guardrails: allowed regions, no disabling of audit logging, no deleting central log buckets, no public storage, required encryption, no leaving the organisation. Test guardrails in a sandbox OU first, because a wrong deny can stop deployments.

Shared capabilities

- **Logging:** audit logs from every account flow to a central, locked log archive account that workload admins cannot change.
- **Networking:** a hub with shared egress, DNS and connectivity to on-premises; spokes connect through it. Plan address ranges across all accounts so they do not overlap.
- **Identity:** people sign in through a central identity provider with single sign-on and get temporary roles per account, with no long-lived users in each account.
- **Security tooling:** detection services enabled in all accounts and sent to one place.

Account vending

New accounts should come from a pipeline: a request creates the account in the right OU, applies baseline guardrails, enables logging, sets budgets and alerts, creates roles and registers it in the inventory. Teams get a safe account in minutes without a ticket queue. Manage the baseline as code, and also plan for closing accounts.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Landing zones and multi-account strategy fits together

Part	What it does	What goes wrong
Why many accounts	An account is a boundary for permissions, limits, billing and blast radius.	One account for all environments.
A typical structure	Keep workloads out of the management account.	Running workloads in the management account.
Guardrails	Policies attached to organisational units limit what even administrators in an account can do.	Guardrails changed without testing.
Shared capabilities	Logging: audit logs from every account flow to a central, locked log archive account that workload admins cannot change.	Overlapping network ranges across accounts.
Account vending	New accounts should come from a pipeline: a request creates the account in the right OU, applies baseline guardrails, enables logging, sets budgets and alerts, creates...	Teams get a safe account in minutes without a ticket queue.

Flow: Why many accounts to A typical structure / A typical structure to Guardrails / Guardrails to Shared capabilities / Shared capabilities to Account vending

Incident: one account, one stolen key

> **Incident:** *A developer's access key leaks and an attacker starts instances for mining in the single account used for everything, including production databases.*

```
$ aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-10-08 --granularity DAILY --me \
    trics UnblendedCost
2026-10-06   412 USD
2026-10-07  9120 USD
```

Because everything shared one account, the key reached production, and the bill could not be traced to a team. The company moved to an organisation with separate accounts for production and development, central logging in a locked account, region guardrails, short-lived single sign-on roles in place of keys, and per-account budget alerts.

Common mistakes

- One account for all environments
- Running workloads in the management account
- Overlapping network ranges across accounts
- Guardrails changed without testing

Check yourself

Why is an account a strong security boundary?

Answer: Permissions, limits and billing are separated, so a compromise stays contained

What does a service control policy or organisation policy do?

Answer: Sets limits that apply to every principal in the accounts below it

Where should audit logs from all accounts be stored?

Answer: In a central account that workload administrators cannot modify

What is account vending?

Answer: Creating new accounts automatically with a secure baseline

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. Object storage and data lifecycle

12 minutes. Use object storage well: classes, lifecycle rules, versioning, access control and protection against deletion and exposure.

Object storage looks like a bottomless folder, and that is why it causes two classic incidents: a bucket left public and a bill that quietly grows for years because nobody deleted old data. A handful of settings (access blocks, versioning, lifecycle rules and encryption) cover most of the risk.

What you will learn

- How object storage differs from a disk
- Storage classes and lifecycle rules that control cost
- Versioning, retention and protection from accidental deletion
- Blocking public access and using signed URLs

Objects, not files

Object storage keeps whole objects (data plus metadata) in a flat namespace, addressed by key such as `reports/2026/01/sales.csv`. The slashes are only part of the name. You cannot edit a part of an object; you replace it. It is very durable (often described as eleven nines), scales without planning, and is accessed over HTTPS. It is not a file system, so it is a poor fit for databases or software that needs fast small writes.

Classes and lifecycle rules

Providers offer classes that trade price for access speed: hot for frequent access, infrequent access for monthly reads, and archive for data you rarely need and can wait hours to get back. Lifecycle rules move or delete objects automatically.

```
{
  "Rules": [{
    "ID": "logs-tiering",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [
      { "Days": 30, "StorageClass": "STANDARD_IA" },
      { "Days": 180, "StorageClass": "GLACIER" }
    ],
    "Expiration": { "Days": 730 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
  }]
}
```

This rule moves logs to a cheaper class after 30 days, to archive after 180, deletes them after two years, and cleans up abandoned uploads. Check minimum storage durations and retrieval fees before you move small or short-lived objects.

Versioning and deletion protection

With **versioning**, an overwrite or delete creates a new version, and the old one stays recoverable. It protects against mistakes and some attacks, but old versions cost money, so add a lifecycle rule for noncurrent versions. For stronger protection use **object lock** (write once, read many) with a retention period, and require multi-factor authentication for delete. Replicate critical buckets to another region or account so a single account problem cannot remove every copy.

Keep it private

Most exposures come from a permissive bucket policy or ACL. Turn on the account-wide **block public access** setting, and give applications access through roles, not shared keys. To share one file for a limited time, create a **signed URL** that expires:

```
$ aws s3 presign s3://acme-reports/2026/01/sales.csv --expires-in 900
```

MARGA by EknathaLabs marga.eknathalabs.com

Encrypt at rest with the provider key or your own key, require HTTPS in the bucket policy, and enable access logging on sensitive buckets. Run the provider's public-access checker regularly and alert when a bucket policy changes.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Object storage and data lifecycle fits together

Part	What it does	What goes wrong
Objects, not files	Object storage keeps whole objects (data plus metadata) in a flat namespace, addressed by key such as reports/2026/01/sales.csv.	Treating object storage like a fast local disk.
Classes and lifecycle rules	Providers offer classes that trade price for access speed: hot for frequent access, infrequent access for monthly reads, and archive for data you rarely need and can...	No lifecycle rule, so data grows forever.
Versioning and deletion protection	With versioning, an overwrite or delete creates a new version, and the old one stays recoverable.	Versioning without expiring old versions.
Keep it private	Most exposures come from a permissive bucket policy or ACL.	Public access allowed on a bucket by accident.

Flow: Objects, not files to Classes and lifecycle rules / Classes and lifecycle rules to Versioning and deletion protection / Versioning and deletion protection to Keep it private

Incident: The bucket that kept growing

Incident: *A finance review found that a logging bucket cost more each month and no one could say why.*

```
$ aws s3 ls s3://acme-logs --recursive --summarize | tail -2
Total Objects: 94,220,118
   Total Size: 412 TiB
$ aws s3api get-bucket-lifecycle-configuration --bucket acme-logs
An error occurred (NoSuchLifecycleConfiguration)
```

Five years of logs sat in the most expensive class, with no expiry rule. Compliance needed only two years. A lifecycle rule moved data to infrequent access at 30 days and to archive at 180, and expired it after two years, which cut the bill by about 70 percent. The team also added a monthly report of the five largest buckets with their growth and owner tags.

Common mistakes

- No lifecycle rule, so data grows forever
- Public access allowed on a bucket by accident
- Versioning without expiring old versions
- Treating object storage like a fast local disk

Check yourself

What does a lifecycle rule do?

Answer: Moves or deletes objects automatically based on age or prefix

What protects against accidental overwrite or delete?

Answer: Versioning, with object lock for stronger protection

How do you share one object temporarily without making it public?

Answer: Create a signed URL that expires

Why is object storage a poor fit for a database?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: You cannot update part of an object, and small writes are slow

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. Connecting networks: peering, VPN and private links

13 minutes. Join cloud networks to each other and to an office or data centre with peering, VPN, dedicated links and private endpoints, and avoid address clashes.

Two teams each built a network in the cloud, both using 10.0.0.0/16. When they try to connect them, nothing works, and changing an address range now means rebuilding everything. Connecting networks is mostly planning: addresses, routes and who may reach what.

What you will learn

- Plan non-overlapping address ranges
- Peering, transit hubs, VPN and dedicated connections
- Private endpoints to reach cloud services without the internet
- Routes and security rules on both sides

Plan addresses first

Networks can only be connected if their ranges do not overlap. Keep one address plan for the whole organization: a large block per cloud account or region, carved into subnets, with a spreadsheet or tool as the record. Reserve space for growth and for the office and data centre ranges. Overlap is the problem that is cheapest to avoid and most expensive to fix, because it can require rebuilding networks or using address translation.

Ways to connect

Option	Use it for	Notes
Peering	Two networks in the cloud	Simple and fast, but not transitive (A to B and B to C does not give A to C)
Transit hub or gateway	Many networks	One central router with route tables; scales better than many peerings
Site-to-site VPN	Office or data centre to cloud	Encrypted over the internet, quick to set up, speed depends on the internet
Dedicated connection	Heavy or steady traffic	A private line from the provider: lower latency, higher cost, weeks to order
Client VPN	Individual engineers	Central access with logging and multi-factor authentication

Use a VPN as the backup for a dedicated line, so a cut fibre does not cut you off.

Private endpoints

Cloud services such as storage or databases are usually reached over public addresses. A **private endpoint** (or interface endpoint) gives the service a private address inside your network, so traffic never leaves the provider network and you can block the internet path entirely. Combine it with a policy that says only your network may use the endpoint. Remember that private DNS must resolve the service name to the private address; otherwise clients still go to the public one.

Routes and rules on both sides

A connection needs three things on each side: a route to the other range, a security rule that allows the traffic, and working DNS. Missing any one makes the link look broken.

```
$ ip route get 172.16.4.10
$ ping -c 2 172.16.4.10
$ nc -zv -w3 172.16.4.10 5432
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ dig +short db.corp.example
```

Check in that order: route, reachability, port, name. Remember that peering and VPN connections need route table entries in every subnet that should use them, and that the other side must route replies back.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Connecting networks: peering, VPN and private... fits together

Part	What it does	What goes wrong
Plan addresses first	Networks can only be connected if their ranges do not overlap.	Connecting with VPN and forgetting that DNS must resolve names on both sides.
Ways to connect	Use a VPN as the backup for a dedicated line, so a cut fibre does not cut you off.	Assuming peering is transitive.
Private endpoints	Cloud services such as storage or databases are usually reached over public addresses.	Letting every team choose its own address range.
Routes and rules on both sides	A connection needs three things on each side: a route to the other range, a security rule that allows the traffic, and working DNS.	Adding a route on one side only.

Flow: Plan addresses first to Ways to connect / Ways to connect to Private endpoints / Private endpoints to Routes and rules on both sides

Incident: The overlapping ranges

Incident: *A migration project needed to connect a new analytics network to the production network, and the connection request was refused.*

```
$ aws ec2 describe-vpcs --query "Vpcs[].[VpcId,CidrBlock]" --output text
vpc-0a1   10.0.0.0/16
vpc-0b2   10.0.0.0/16
```

Both networks used 10.0.0.0/16, and the provider cannot peer overlapping ranges. The analytics network was rebuilt in 10.40.0.0/16 over a weekend, and the organization introduced an address plan with a reviewed registry. New networks now get their range from the registry, and a pipeline check rejects any VPC definition whose range overlaps an existing one.

Common mistakes

- Letting every team choose its own address range
- Assuming peering is transitive
- Adding a route on one side only
- Connecting with VPN and forgetting that DNS must resolve names on both sides

Check yourself

Why must connected networks avoid overlapping ranges?

Answer: Routers cannot tell which network an address belongs to

Is cloud peering transitive?

Answer: No, each peering links only two networks

What is a private endpoint for?

Answer: Reaching a cloud service by a private address, without the internet

Which check order helps find a broken connection?

Answer: Route, reachability, port, name

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 15. Debugging cloud problems: permissions, networking and quotas

15 minutes. Trace cloud failures through identity, network path, quotas and service health using audit logs, policy simulation and flow logs instead of guessing.

Cloud problems often look like application bugs. A request times out, an upload fails, or a deploy is denied. The causes are usually one of four things: an identity policy, a network rule, a quota or limit, or a service problem on the provider side. Each leaves a record if you know where to look.

What you will learn

- Reading "access denied" correctly
- Network path checks: security groups, route tables, network ACLs, flow logs
- Quotas and limits
- Provider status and who changed what

Identity: who was denied, and why

An AccessDenied error names the principal (the role or user), the action and the resource. Check them in order:

1. Is the caller the identity you think it is?
2. Does an identity policy allow the action on that resource?
3. Does a resource policy, permissions boundary, organisation policy or explicit deny override it?

```
aws sts get-caller-identity                              # which identity am I?
aws iam simulate-principal-policy --policy-source-arn ROLE_ARN \
  --action-names s3:GetObject --resource-arns arn:aws:s3:::shop-data/*
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=GetObject - \
    -max-results 5
```

Audit logs show the exact denied call with the error code and the calling role.

Network: follow the packet

Check	What to look for
Security group	Inbound and outbound rules on both ends; the source and port
Route table	A route to the destination (gateway, NAT, peering, endpoint)
Network ACL	Stateless rules: both directions must allow
DNS	Private zone attached to the right network
Flow logs	REJECT records show who was blocked

A private instance with no route to a NAT gateway cannot reach package repositories, and the error is a plain timeout.

Quotas and capacity

"Limit exceeded", "InsufficientInstanceCapacity" and throttling errors are not bugs in your code. Check service quotas, request an increase early, and add retries with backoff and jitter. Capacity errors for one instance type in one zone can be avoided by allowing several types and zones.

Who changed what

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=ResourceName,AttributeValue=sg-0ab \
    c --max-results 3
EventName: RevokeSecurityGroupIngress   Username: deploy-role   EventTime: 09:12
```

MARGA by EknathaLabs marga.eknathalabs.com

A change record near the start of an outage is the strongest lead. Also check the provider's status page and your account's health notices, but confirm with your own metrics.

Diagram: Where a cloud request can be stopped

Part	What it does	What goes wrong
Identity	Credentials, role and policies decide whether the call is allowed.	Wrong role, explicit deny or boundary gives AccessDenied.
Network path	Security groups, routes, ACLs and endpoints decide whether packets arrive.	A missing route or rule shows as a timeout.
Quota and capacity	Service limits and regional capacity cap what you can create.	Limit errors look like random API failures.
Service health	The provider service itself may be degraded.	Blaming your code while the provider has an incident.
Audit and flow logs	CloudTrail-style logs and flow logs show what happened and who did it.	Logging that was never enabled leaves no evidence.

Flow: Identity to Audit and flow logs / Network path to Audit and flow logs / Quota and capacity to Audit and flow logs / Service health to Audit and flow logs

Incident: the deployment that lost S3 access

> **Incident:** *At 14:05 the nightly job begins failing with AccessDenied on a storage bucket it has used for a year.*

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=PutBucket \
    Policy --max-results 1
Username: platform-admin   EventTime: 14:02
$ aws s3api get-bucket-policy --bucket shop-data | jq -r .Policy | grep -i -B2 -A4 Deny
"Effect":"Deny","Principal":"*","Condition":{"StringNotEquals":{"aws:SourceVpce":"vpce-0abc"}}
```

Three minutes earlier a new bucket policy denied access except through one VPC endpoint. The job ran from another network, so it was blocked. The team added the second endpoint to the policy, and made bucket policy changes go through review with a simulation step.

Common mistakes

- Granting broad permissions "to test"
- Ignoring explicit denies
- No audit logging or flow logs when the incident happens
- Retrying without backoff, which worsens throttling

Check yourself

What is the first thing to confirm for an AccessDenied error?

Answer: Which identity made the call, with get-caller-identity or the audit log

A connection from a private subnet times out. What is likely?

Answer: A missing route (for example NAT) or a blocking rule

Where do you find who changed a security group and when?

Answer: The audit log (CloudTrail or equivalent)

How should clients handle throttling errors?

Answer: Retry with exponential backoff and jitter

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 16. FinOps: reading a cloud bill and tagging for ownership

12 minutes. Break a cloud bill down by service, account and tag, understand how usage turns into charges, and set up tags and accounts so every dollar has an owner.

Cloud cost is an engineering metric like latency: it changes with every deploy and nobody notices until the bill arrives. FinOps is the habit of making cost visible, owned and discussed by the people who can change it. The first skill is simply reading the bill.

What you will learn

- How usage becomes line items on a bill
- Group costs by service, account, region and tag
- Design a small, enforced tagging scheme
- Tell unit cost from total cost

How the bill is built

Every resource creates **usage** (instance hours, GB stored, requests, GB transferred), and each usage type has a price. Your bill is the sum of usage times price, minus discounts, plus tax. Providers expose the raw data: AWS Cost and Usage Reports and Cost Explorer, Azure Cost Management, and Google Cloud Billing export. Start with the service view, then drill into account, region, usage type and tag.

```
$ aws ce get-cost-and-usage --time-period Start=2026-03-01,End=2026-04-01 --granularity MONTHLY -- \
    metrics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE
$ aws ce get-cost-and-usage --time-period Start=2026-03-01,End=2026-04-01 --granularity DAILY --me \
    trics UnblendedCost --group-by Type=TAG,Key=team
```

Grouping by day shows when a cost jumped, which you can match to a deploy or a change. Grouping by a tag shows who spends it.

Accounts and tags give ownership

Two tools assign cost to an owner. **Accounts, subscriptions or projects** are hard boundaries: one per environment or per team makes cost, permissions and limits clear. **Tags or labels** are flexible: key-value pairs such as `team`, `service`, `env` and `cost-center`. Keep the set small and mandatory, and enforce it, because optional tags decay.

```
required_tags:
  - team
  - service
  - env
allowed_env: [dev, staging, prod]
```

Enforce through infrastructure as code (a module that refuses resources without tags), policy checks in CI, and cloud policies that deny creation or report non-compliance. Remember that some costs are not taggable (shared networking, support), so allocate them with an agreed rule, such as proportional to usage.

Total cost and unit cost

A bill that grows 30 percent is bad news only if the business grew less. Track **unit cost**: cost per order, per active user, per thousand requests or per build. Rising total cost with falling unit cost is healthy growth; rising unit cost means waste or a design problem. Put unit cost on the same dashboard as latency and error rate.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How FinOps: reading a cloud bill and tagging for... fits together

Part	What it does	What goes wrong
How the bill is built	Every resource creates usage (instance hours, GB stored, requests, GB transferred), and each usage type has a price.	Looking only at the monthly total.
Accounts and tags give ownership	Two tools assign cost to an owner. Accounts, subscriptions or projects are hard boundaries: one per environment or per team makes cost, permissions and limits clear.	Optional tags that nobody follows.
Total cost and unit cost	A bill that grows 30 percent is bad news only if the business grew less.	Not tracking unit cost.

Flow: How the bill is built to Accounts and tags give ownership / Accounts and tags give ownership to Total cost and unit cost

Incident: The mystery line

Incident: *The finance team noticed a 38 percent rise in the monthly bill and nobody in engineering could say why.*

```
$ aws ce get-cost-and-usage --time-period Start=2026-03-01,End=2026-03-15 --granularity DAILY --me \
    trics UnblendedCost --group-by Type=DIMENSION,Key=USAGE_TYPE
```

The daily view showed a step on March 9 in a data transfer usage type, and the tag view showed 40 percent of that spend had no `team` tag. The cause was a new analytics job copying data between regions every hour. Because the resources were untagged, nobody was alerted. The team tagged the job, moved processing next to the data, made tags mandatory in their Terraform modules and added a weekly review of untagged spend.

Common mistakes

- Looking only at the monthly total
- Optional tags that nobody follows
- No account or project structure to separate environments
- Not tracking unit cost

Check yourself

What is a bill line item?

Answer: Usage of a resource type multiplied by its price

Why group cost by day?

Answer: To see when a jump started and link it to a change

What makes tags useful for ownership?

Answer: A small mandatory set that is enforced automatically

What does unit cost show?

Answer: Cost per business unit such as an order or a request

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 17. FinOps: rightsizing and removing waste

12 minutes. Find idle and oversized resources with utilization data, retire unattached disks and old snapshots safely, and schedule non-production environments off hours.

The cheapest resource is the one you do not run. Typical clouds hold a long tail of waste: instances sized for a peak that never came, disks left behind after a delete, load balancers with no targets, and test environments that run all weekend. Removing waste needs data and a safe process, not a purge.

What you will learn

- Rightsize compute from real utilization
- Find unattached volumes, old snapshots and idle load balancers
- Schedule development and test environments
- Remove things safely with owners and a grace period

Rightsize from data

Look at utilization over a representative period (at least two weeks, and include month-end if the workload has one). For CPU, look at the high percentile (p95 or p99), not the average, and for memory the working set including headroom. An instance that never goes above 15 percent CPU and uses a quarter of its memory is a candidate for a smaller size or a different family. Cloud tools (AWS Compute Optimizer, Azure Advisor, Google Cloud recommender) produce suggestions; treat them as a starting point and check burst patterns, licensing and network limits that depend on size.

```
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization --dimensio \
    ns Name=InstanceId,Value=i-0abc123 --start-time 2026-03-01T00:00:00Z --end-time 2026-03-15T00: \
    00:00Z --period 3600 --statistics Maximum
$ aws ec2 describe-volumes --filters Name=status,Values=available --query 'Volumes[].[VolumeId,Siz \
    e,CreateTime]' --output table
```

Change one step at a time (one size down), watch latency and errors, and keep a quick path back.

The waste checklist

- **Unattached volumes and old snapshots**: left after instances were deleted. Snapshot lifecycle policies should expire them.
- **Idle load balancers, NAT gateways and IP addresses**: they bill by the hour even with no traffic.
- **Oversized databases and caches**: provisioned for a peak, with low connections and CPU.
- **Orphaned test stacks**: forgotten feature environments.
- **Over-retained logs and metrics**: ingest is often the larger cost than storage.

Each item should have an owner via tags. Items without an owner enter a process: notify, wait a fixed time, snapshot, then delete.

Schedule what does not need to run

Development, test and demo environments often need to run only during working hours, which is about a quarter of the week. Stop instances, scale node groups to zero and pause databases outside those hours, and make it easy to start them again. Automate with the provider's scheduler, a pipeline job or a small script, and tag environments with their schedule.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How FinOps: rightsizing and removing waste fits together

Part	What it does	What goes wrong
Rightsize from data	Look at utilization over a representative period (at least two weeks, and include month-end if the workload has one).	Sizing from average CPU instead of high percentiles.
The waste checklist	Unattached volumes and old snapshots: left after instances were deleted.	Deleting "unused" resources without owners or a grace period.
Schedule what does not need to run	Development, test and demo environments often need to run only during working hours, which is about a quarter of the week.	Optimizing small items while ignoring a large database or data transfer.

Flow: Rightsize from data to The waste checklist / The waste checklist to Schedule what does not need to run

Incident: The cleanup that deleted a restore

Incident: *A cost cleanup removed "unused" snapshots older than 90 days, and a month later a team needed one of them to recover a customer database.*

```
$ aws ec2 describe-snapshots --owner-ids self --query 'Snapshots[?StartTime<=`2026-01-01`].[Snapsh \
    otId,Description]' --output text | head -3
snap-0a1  quarterly-archive-prod-db
```

The snapshots were the only copies of quarterly archives, but they were untagged and looked abandoned. The clean-up job had no grace period and no review. Recovery took days, from a partial backup. The team now requires an owner and a retention tag on every snapshot, runs cleanup in report mode first, adds a 14-day quarantine (stop or snapshot before delete) and keeps a short list of "never delete without a human" resource types.

Common mistakes

- Sizing from average CPU instead of high percentiles
- Deleting "unused" resources without owners or a grace period
- Optimizing small items while ignoring a large database or data transfer
- Rightsizing production in one big jump

Check yourself

Which metric is better for sizing CPU?

Answer: A high percentile such as p95 over a representative period

Why quarantine before deleting?

Answer: Wrongly identified resources can be restored

Which resources bill hourly even with no traffic?

Answer: Load balancers, NAT gateways and reserved IP addresses

How can you reduce the cost of dev and test environments?

Answer: Stop or scale them to zero outside working hours

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 18. FinOps: commitments, savings plans and spot capacity

12 minutes. Lower the unit price of steady workloads with reserved and committed discounts, run fault-tolerant work on spot capacity, and avoid over-committing.

After you remove waste and rightsize, the next lever is the price you pay per hour. Providers give lower prices in exchange for commitment (steady use over one or three years) or flexibility (capacity they can take back). Used well, these cut a large share of compute cost. Used badly, they lock you into payments for things you no longer run.

What you will learn

- The main pricing models: on-demand, commitments and spot
- How to choose a commitment level from your baseline
- Which workloads suit spot and how to make them survive interruption
- Risks and how to manage a commitment portfolio

Pricing models

- **On-demand**: pay as you go, no commitment, highest unit price. Right for unpredictable or short-lived use.
- **Commitments**: reserved instances, savings plans (AWS), reservations and savings plans (Azure) and committed use discounts (Google Cloud). You promise a level of use, or a spend per hour, for one or three years and get a lower rate. Flexible plans apply across instance families or services; narrow ones give bigger discounts but fit less.
- **Spot or preemptible**: unused capacity at a deep discount, which the provider can reclaim with short notice (often two minutes or less).

Exact discounts and terms change, so check the current provider pages before you decide.

Commit to the baseline, not the peak

Plot hourly usage over several months. The **always-on floor** is what you can commit to. The variable part above it stays on-demand or spot. If you commit to the average or the peak, you pay for unused commitment during quiet periods. Start with a smaller share of the floor (for example 50 to 70 percent), observe **coverage** (how much usage is discounted) and **utilization** (how much of the commitment is used), and add more in steps. Stagger purchase dates so that not everything expires at once, and review before each expiry, since your architecture may have changed (for example a move to containers, a new instance family or another region).

Spot for fault-tolerant work

Good fits: batch jobs, CI runners, stateless web workers behind a load balancer, data processing with checkpoints, and Kubernetes node pools for scalable workloads. Poor fits: single databases, anything that cannot be interrupted or restarted.

```
apiVersion: v1
kind: Pod
metadata:
  name: batch-worker
spec:
  terminationGracePeriodSeconds: 90
  tolerations:
    - key: "capacity"
      operator: "Equal"
      value: "spot"
      effect: "NoSchedule"
  nodeSelector:
    capacity: spot
```

MARGA by EknathaLabs marga.eknathalabs.com

```
containers:
  - name: worker
    image: registry.example.com/batch-worker:1.4.2
```

Design for interruption: handle the termination notice (drain, checkpoint, return work to the queue), spread across several instance types and zones so one pool drying up does not stop you, and keep a small on-demand base so the service stays up when spot is scarce.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How FinOps: commitments, savings plans and spot... fits together

Part	What it does	What goes wrong
Pricing models	On-demand: pay as you go, no commitment, highest unit price.	Buying everything at once with the same expiry date.
Commit to the baseline, not the peak	Plot hourly usage over several months. The always-on floor is what you can commit to.	Not watching coverage and utilization after the purchase.
Spot for fault-tolerant work	Good fits: batch jobs, CI runners, stateless web workers behind a load balancer, data processing with checkpoints, and Kubernetes node pools for scalable workloads.	Running stateful systems on spot.

Flow: Pricing models to Commit to the baseline, not the peak / Commit to the baseline, not the peak to Spot for fault-tolerant work

Incident: Paying for servers that were gone

> **Incident:** *A company bought a three-year commitment for a specific instance family, then migrated to containers on a different family six months later.*

```
$ aws ce get-reservation-utilization --time-period Start=2026-02-01,End=2026-03-01
Utilization: 38%
```

The commitment utilization fell to 38 percent, so the company paid for capacity it did not use while also paying on-demand for the new family. The team sold what they could on the marketplace, converted part to flexible plans, and changed their process: commitments are bought in steps, reviewed quarterly against the roadmap, owned by a named person and limited to a share of the baseline.

Common mistakes

- Committing to the peak or the average
- Buying everything at once with the same expiry date
- Running stateful systems on spot
- Not watching coverage and utilization after the purchase

Check yourself

What should a commitment normally cover?

Answer: The steady baseline of usage

What is the risk of spot capacity?

Answer: The provider can reclaim it on short notice

Which workload suits spot?

Answer: A stateless batch worker with checkpoints

Why review commitments before they expire?

Answer: Architecture and demand may have changed

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 19. FinOps: Kubernetes cost allocation and resource requests

13 minutes. Show which team and workload costs what in a shared cluster, set requests and limits from real usage, and use autoscaling and node choices to cut idle capacity.

In a shared cluster the cloud bill shows nodes, not teams. A pod that asks for 8 CPUs and uses 0.3 still blocks the capacity it requested. Kubernetes cost work therefore starts with requests, because the scheduler packs by what pods ask for, not by what they use.

What you will learn

- Why requests, not usage, decide how many nodes you pay for
- Allocate node cost to namespaces and workloads
- Set requests from observed usage and use autoscalers
- Find idle capacity and fix bin-packing problems

Requests drive node count

The scheduler places a pod on a node if the node has enough **unrequested** CPU and memory. If teams copy large requests "to be safe", nodes look full while real utilization is low, and the cluster autoscaler adds more nodes. The gap between requested and used resources is waste you pay for. Compare the two for each workload.

```
$ kubectl top pods -A --sort-by=cpu | head
$ kubectl get pods -A -o custom-columns=NS:.metadata.namespace,POD:.metadata.name,CPUREQ:.spec.con \
    tainers[*].resources.requests.cpu,MEMREQ:.spec.containers[*].resources.requests.memory | head
$ kubectl describe node node-a | grep -A6 "Allocated resources"
```

`kubectl top` needs the metrics server and shows current use only. For decisions use a longer window from your monitoring system (for example the 95th percentile over two weeks).

Allocate cost to teams

Split node cost by what each workload reserves (requests), blended with usage where available. Group by the labels `team`, `service` and `env` that you require on every namespace and workload. Tools such as OpenCost, Kubecost and the cloud providers' own allocation features calculate this, and add the shared and idle parts: unrequested node capacity is **idle cost**, spread across teams by an agreed rule or reported separately. Show each team their share weekly, next to their unit cost.

Set requests and limits well

- **Memory**: set the request close to the working set plus headroom, and a limit at or slightly above it, since memory cannot be shared by throttling and over-use means a kill.
- **CPU**: set a request at typical load. Be careful with tight CPU limits, which can throttle latency-sensitive services even when the node has spare CPU. Many teams set requests but no CPU limit.
- Use the **Vertical Pod Autoscaler** in recommendation mode to learn sensible values, and the **Horizontal Pod Autoscaler** to change replica counts with load.
- Use **ResourceQuotas** and **LimitRanges** per namespace so one team cannot reserve the whole cluster.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-checkout
  namespace: checkout
spec:
  hard:
    requests.cpu: "40"
    requests.memory: 160Gi
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    limits.memory: 200Gi
```

Nodes and packing

Mix node sizes to fit pod shapes, use autoscalers that choose instance types for pending pods (such as Karpenter), consolidate underused nodes, and run fault-tolerant workloads on spot pools. Pod disruption budgets make consolidation safe. Look at DaemonSets as well: ten small agents on every node add up on big fleets.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How FinOps: Kubernetes cost allocation and resource... fits together

Part	What it does	What goes wrong
Requests drive node count	The scheduler places a pod on a node if the node has enough unrequested CPU and memory.	Large requests "just in case".
Allocate cost to teams	Split node cost by what each workload reserves (requests), blended with usage where available.	Tight CPU limits that throttle a latency-sensitive service.
Set requests and limits well	Memory: set the request close to the working set plus headroom, and a limit at or slightly above it, since memory cannot be shared by throttling and over-use means a...	No quotas, so one namespace takes the cluster.
Nodes and packing	Mix node sizes to fit pod shapes, use autoscalers that choose instance types for pending pods (such as Karpenter), consolidate underused nodes, and run fault-tolerant...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Requests drive node count to Allocate cost to teams / Allocate cost to teams to Set requests and limits well / Set requests and limits well to Nodes and packing

Incident: Forty nodes for the work of twelve

Incident: *A cluster grew from 12 to 40 nodes over a quarter, while traffic grew by only 20 percent.*

```
$ kubectl describe node ip-10-0-3-41 | grep -A5 "Allocated resources"
  cpu      15700m (98%)
  memory   52Gi (83%)
$ kubectl top node ip-10-0-3-41
cpu 2100m (13%)   memory 21Gi (33%)
```

Teams had copied a template with 4 CPU and 8 GiB requests per pod, and pods used a fraction of it. The nodes were full on paper and idle in practice. The team ran a recommendation pass with VPA, lowered requests by workload in steps, added quotas and a monthly cost report per namespace, and the cluster settled at 16 nodes with the same traffic and no new incidents.

Common mistakes

- Large requests "just in case"
- Reading `kubectl top` once and deciding
- Tight CPU limits that throttle a latency-sensitive service
- No quotas, so one namespace takes the cluster

Check yourself

What decides how many pods fit on a node?

Answer: The pods' resource requests

What is idle cost in a cluster?

Answer: The cost of node capacity that nobody has requested

MARGA by EknathaLabs
marga.eknathalabs.com

What is a risk of a very low CPU limit?

Answer: The service may be throttled and slow even if the node is idle

How do you find sensible request values?

Answer: Measure usage over a long window and add headroom, for example with VPA recommendations

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 20. FinOps: data transfer, storage tiers and hidden costs

12 minutes. Spot the costs that surprise teams, such as cross-zone and egress traffic, NAT gateways, log ingestion and storage classes, and design to reduce them.

Compute is easy to see. The surprises are in the less visible lines: data moving between zones and out to the internet, a NAT gateway that processes every byte, logs billed per gigabyte, and storage kept at the most expensive tier. These costs scale with traffic and design choices, so architecture reviews are the best place to control them.

What you will learn

- Which data transfer paths are billed and which are usually free
- Why NAT gateways, load balancers and cross-zone traffic add up
- Storage classes, lifecycle rules and retention
- Observability and logging cost control

Data transfer paths

Rules differ by provider and change over time, so check the current price pages, but the pattern is stable:

- Inbound traffic from the internet is usually free.
- Outbound traffic (**egress**) to the internet is billed per GB, often at the highest rate.
- Traffic **between regions** is billed, and **between availability zones** is often billed in each direction.
- Traffic inside one zone over private addresses is usually free.
- Using a CDN can cut origin egress and improve latency.

So an architecture that spreads chatty services across three zones, or replicates data to another region, pays for every byte that crosses. That can still be the right choice for resilience, but it should be a conscious one.

NAT gateways and managed services

A managed NAT gateway charges an hourly fee and a per-GB processing fee on all traffic through it. Services that pull large images, packages or backups from the internet or from provider services through NAT can create a very large bill. Mitigations: use **VPC gateway or private endpoints** for storage and registry traffic, cache images and packages inside the network, and place chatty components in the same zone when resilience allows (zone-aware routing in Kubernetes can reduce cross-zone calls).

```
$ aws ce get-cost-and-usage --time-period Start=2026-03-01,End=2026-04-01 --granularity MONTHLY -- \
    metrics UnblendedCost --filter '{"Dimensions":{"Key":"USAGE_TYPE_GROUP","Values":["EC2: Data T \
    ransfer - Inter AZ"]}}'
```

Storage tiers and lifecycle

Object storage has classes from hot to archive, with lower price per GB but retrieval fees, minimum storage periods and slower access. Use lifecycle rules to move data by age, expire temporary data and abort incomplete multipart uploads. Turn on versioning only with a rule to expire old versions. For block volumes, delete after use, choose the volume type by actual IOPS need, and expire old snapshots.

```
Rules:
  - ID: tier-and-expire-logs
    Status: Enabled
    Filter:
      Prefix: logs/
    Transitions:
      - Days: 30
        StorageClass: STANDARD_IA
    Expiration:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
      Days: 365
    AbortIncompleteMultipartUpload:
      DaysAfterInitiation: 7
```

Logs, metrics and traces

Ingest volume is often the biggest part of observability cost. Reduce it by dropping debug logs in production, sampling traces, limiting metric cardinality (labels with unbounded values such as user ids multiply series), and setting retention by value: short for verbose data, long only for audit logs. Put costs per team on the dashboard so teams see what their logging costs.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How FinOps: data transfer, storage tiers and hidden... fits together

Part	What it does	What goes wrong
Data transfer paths	Rules differ by provider and change over time, so check the current price pages, but the pattern is stable.	Ignoring cross-zone and NAT charges in the design.
NAT gateways and managed services	A managed NAT gateway charges an hourly fee and a per-GB processing fee on all traffic through it.	Keeping all data in the hottest storage class.
Storage tiers and lifecycle	Object storage has classes from hot to archive, with lower price per GB but retrieval fees, minimum storage periods and slower access.	Logging everything at debug level with long retention.
Logs, metrics and traces	Ingest volume is often the biggest part of observability cost.	Unbounded metric labels.

Flow: Data transfer paths to NAT gateways and managed services / NAT gateways and managed services to Storage tiers and lifecycle / Storage tiers and lifecycle to Logs, metrics and traces

Incident: The NAT bill

Incident: *A small company saw its networking line become larger than its compute after a nightly job was moved into private subnets.*

```
$ aws ce get-cost-and-usage --time-period Start=2026-03-01,End=2026-03-10 --granularity DAILY --me \
    trics UnblendedCost --filter '{"Dimensions":{"Key":"USAGE_TYPE","Values":["EUW1-NatGateway-Byt \
    es"]}}'
```

The job pulled several terabytes per night from object storage in the same region, and the traffic went through a NAT gateway that charged per processed GB. A gateway endpoint for the storage service made that path private and free of NAT processing fees. The team also added a check in design reviews for large flows, and a budget alert on the NAT usage type.

Common mistakes

- Ignoring cross-zone and NAT charges in the design
- Keeping all data in the hottest storage class
- Logging everything at debug level with long retention
- Unbounded metric labels

Check yourself

Which traffic is usually billed?

Answer: Egress to the internet and traffic between regions or zones

Why can a NAT gateway be expensive?

Answer: It charges per GB processed as well as per hour

MARGA by EknathaLabs marga.eknathalabs.com

What do lifecycle rules do for storage?

Answer: Move or expire data by age automatically

What often multiplies metric cost?

Answer: High-cardinality labels such as user IDs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 21. FinOps: budgets, anomaly alerts and a cost practice

11 minutes. Set budgets and anomaly alerts, run a regular cost review with engineering and finance, and build cost into design reviews and pipelines so savings last.

One-off savings fade. The lasting result comes from a small, boring routine: budgets that alert, a regular review with owners, cost estimates in design and pipelines that show the effect of a change before it ships.

What you will learn

- Budgets and anomaly detection and how to route alerts
- A weekly and monthly review rhythm
- Cost in design reviews and pull requests
- The roles of engineering, finance and leadership

Budgets and anomalies

A **budget** is a planned amount per account, team or tag. Set alerts on actual spend (for example at 50, 80 and 100 percent) and on **forecasted** spend, which warns you before the month ends. An **anomaly detector** flags unusual changes in a service or usage type, even when the budget is not near. Route alerts to the owner's team channel, not just to finance, with a link to the cost view for that service, so the person who can act sees it first.

```
$ aws budgets describe-budgets --account-id 111122223333
$ aws ce get-anomalies --date-interval StartDate=2026-03-01,EndDate=2026-03-15
```

Tune thresholds so alerts are rare and meaningful; an alert that fires every day is ignored.

A review rhythm

- **Weekly, 15 minutes per team**: top five changes in cost, untagged spend, new anomalies, the next action and its owner.
- **Monthly with finance**: actual against forecast, unit cost trends, commitment coverage and utilization, planned savings and their status.
- **Quarterly**: architecture topics (data transfer, storage tiers, regions), commitment renewals, targets.

Write down actions with owners and dates. Celebrate wins in public, since people repeat what gets noticed.

Cost in design and delivery

Add a **cost section** to design documents: expected traffic, major cost drivers, a rough monthly estimate and what could surprise us. In pull requests for infrastructure code, show the estimated cost change (tools such as Infracost can do this for Terraform). Add guardrails: default instance sizes, allowed regions, required tags, budgets created together with new accounts, and limits on expensive services in non-production.

```
budget_policy:
  new_account:
    monthly_budget: 2000
    alerts: [50, 80, 100]
    forecast_alert: true
    required_tags: [team, service, env]
  non_prod:
    allowed_regions: [eu-west-1]
    schedule: office-hours
```

MARGA by EknathaLabs marga.eknathalabs.com

Who does what

Engineering owns the usage and the design. Finance owns forecasting, allocation rules and reporting. Leadership sets targets and removes blockers. A small FinOps group (often part of the platform team) builds the tooling and the reports and coaches teams. Measure the practice by unit cost trends, tag coverage, commitment utilization and the share of spend that has an owner, not only by the total.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How FinOps: budgets, anomaly alerts and a cost... fits together

Part	What it does	What goes wrong
Budgets and anomalies	A budget is a planned amount per account, team or tag.	One budget alert at 100 percent.
A review rhythm	Weekly, 15 minutes per team: top five changes in cost, untagged spend, new anomalies, the next action and its owner.	Alerts sent only to finance.
Cost in design and delivery	Add a cost section to design documents: expected traffic, major cost drivers, a rough monthly estimate and what could surprise us.	Treating cost as a once-a-year clean-up.
Who does what	Engineering owns the usage and the design.	Reviews that produce no owned actions.

Flow: Budgets and anomalies to A review rhythm / A review rhythm to Cost in design and delivery / Cost in design and delivery to Who does what

Incident: The weekend loop

> **Incident:** *A misconfigured retry loop in a data job wrote the same objects millions of times over a long weekend, and the bill arrived three weeks later.*

```
$ aws ce get-cost-and-usage --time-period Start=2026-03-07,End=2026-03-10 --granularity DAILY --me \
    trics UnblendedCost --group-by Type=DIMENSION,Key=SERVICE
```

There was a monthly budget with a single alert at 100 percent, sent to a finance mailbox, and no anomaly detection. The loop cost several times a normal month's storage line. The team added daily anomaly alerts routed to owners, a forecast alert at 80 percent, request-rate limits on the job and a weekly review. Two months later a similar bug was caught within five hours.

Common mistakes

- One budget alert at 100 percent
- Alerts sent only to finance
- Reviews that produce no owned actions
- Treating cost as a once-a-year clean-up

Check yourself

What does a forecast alert add to a budget?

Answer: A warning before the month ends if spend is projected to exceed the budget

Who should receive a cost anomaly alert?

Answer: The team that owns the service

What belongs in a design document's cost section?

Answer: Expected cost drivers, a monthly estimate and likely surprises

Which measures show a healthy FinOps practice?

Answer: Unit cost trend, tag coverage and commitment utilization

MARGA by EknathaLabs marga.eknathalabs.com